

Learning Elixir from Claude

A readable companion to the official documentation · checked against Elixir 1.20

Anatoly Rodionov, with Claude

Contents

[Introduction](#)

[1. Pattern Matching and Data](#)

[2. Errors and Control Flow](#)

[3. Processes, Tasks and Agents](#)

[4. GenServer](#)

[5. Supervision and Registry](#)

[6. Enum, Stream and Collections](#)

[7. Strings and Unicode](#)

[8. Protocols and Behaviours](#)

[9. Macros](#)

[10. Mix, Configuration and Releases](#)

[11. Testing with ExUnit](#)

[12. Types](#)

[13. Observability](#)

[14. Review Cheat Sheet](#)

Introduction

Learning Elixir from Claude · checked against Elixir 1.20 · Sep 26, 2026

Why this book exists

AI assistants now write a large share of everyday Elixir. They are fluent in the syntax, the pipe operator and the standard library. What they get wrong is quieter: a `map[:key]` where a missing key should have failed loudly, a `GenServer` that exists only to group functions, a `Task.async` that takes its caller down with it, a config value frozen at compile time.

That code compiles, reads well, and passes the obvious test. It fails later, under load, on unexpected input, or in the release rather than in `ixex`. You can only catch it if you know what the language and the runtime actually do.

This book is the Elixir half of a pair. Its companion, [Learning Erlang from Claude](#), covers the BEAM runtime itself: processes and mailboxes, binaries and memory, ETS, distribution, schedulers. Everything there applies to Elixir unchanged, so this book links to it instead of repeating it, and spends its pages on what Elixir adds.

Where the material comes from

The Elixir documentation is unusually good. Beyond the API reference it includes guides and a set of official *anti-pattern* pages covering code, design, processes and macros. This companion reorganises that material around concepts, checks it against runnable code, and adds the section reference docs never have: code that looks right and isn't.

How each chapter is built

1. **Why it exists.** The concept, and the problem it solves.
2. **The building blocks.** The API, condensed into tables.
3. **How it actually works.** What the compiler and runtime do underneath.
4. **Plausible but wrong.** Code that compiles and passes a happy-path test, why it fails, and the fix.
5. **Review checklist and sources.** What to check in real code, with links.

The “Plausible but wrong” section is the heart of each chapter. Its examples were run on a real Elixir installation wherever the behaviour could be shown in a script.

How to read it

Part	Chapters	Read when
The language	1 Pattern Matching and Data · 2 Errors and Control Flow	First

Part	Chapters	Read when
Concurrency	3 Processes, Tasks and Agents · 4 GenServer · 5 Supervision and Registry	After part 1
Working with data	6 Enum, Stream and Collections · 7 Strings and Unicode · 8 Protocols and Behaviours	Any time after part 1
Building projects	9 Macros · 10 Mix, Config and Releases · 11 Testing with ExUnit · 12 Types	Before shipping
Running it	13 Observability	In production
Reference	14 Review Cheat Sheet	When reviewing code

If you come from Erlang, chapter 12 of the Erlang book maps the two languages; start there, then read this book from chapter 1.

Using it with an AI assistant

- **As a review list.** The cheat sheet collects every checklist. Run generated code past it before accepting it.
- **As a prompt.** Ask the assistant to check its code against a chapter’s “Plausible but wrong” section, or to write the test that would expose a given failure.
- **As a check on the assistant.** When it claims something about how Elixir behaves, each chapter’s sources show where to verify it.

Conventions

- Facts were checked against the Elixir 1.20 documentation. Examples were run on Elixir 1.14 and again on Elixir 1.18 (OTP 25), with the same results; where behaviour differs between versions, the text says so.
- Code examples are short and show one mistake each.
- “The Erlang book” means the companion *Learning Erlang from Claude*, cited by chapter number.

A note on accuracy

This book was written with Claude, an AI assistant, and checked against the official documentation it cites. It can still contain mistakes. Treat the linked documentation as the authority, and please report errors as issues on the repository.

1. Pattern Matching and Data

Chapter 1 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why pattern matching is the core of Elixir

In most languages, `=` assigns. In Elixir it *matches*: the left side is a pattern, and the match either succeeds, binding variables, or raises. That one idea runs through the whole language: function heads, `case`, `with`, `receive`, and the way you are expected to handle data you don't trust.

It also sets the style. Elixir code is meant to be *assertive*: state the shape you expect and let anything else fail immediately, close to its cause. The official anti-pattern guide lists the opposite, *non-assertive* code, as a named anti-pattern three times over. Much plausible-but-wrong Elixir is defensive code written with habits from other languages: returning `nil` "to be safe", reading a map with `[]` "in case the key is missing", accepting anything truthy.

2. The building blocks

Matching

Construct	Meaning
<code>{:ok, v} = f()</code>	Match or raise <code>MatchError</code> ; binds <code>v</code>
<code>^x</code>	Compare with the existing value of <code>x</code> instead of rebinding
<code>def f(%{id: id})</code>	Match in a function head; next clause if it fails
<code>when guard</code>	Extra condition on a clause, from a restricted set of expressions
<code>_</code> / <code>_name</code>	Match anything without binding (or bind and mark unused)

Guards allow only a fixed set of expressions: comparisons, `and` / `or` / `not`, arithmetic, type checks like `is_map/1`, and some built-ins like `map_size/1`. `&&`, `||` and `!` are not allowed, because they are not strictly boolean.

Choosing a data structure

Structure	Keys	Order	Duplicates	Access	Use for
Map <code>%{}</code>	Any term	Not guaranteed	No	Logarithmic time	General key-value data
Struct <code>%User{}</code>	Fixed atom set	—	No	<code>user.field</code>	Domain data with a known shape

Structure	Keys	Order	Duplicates	Access	Use for
Keyword list [a: 1]	Atoms	Kept	Allowed	Linear scan	Options passed to functions
Tuple {:ok, v}	Positional	Fixed	—	By pattern	Small fixed groupings, tagged results

A struct is a map with an extra `__struct__` field. It does not implement `Access` (so no `user[:name]`) or `Enumerable`.

Two ways to read a map

The docs draw the line clearly: `map.key` is for maps with “a predetermined set of atom keys, which are expected to always be present,” and raises `KeyError` if one is missing. `map[key]` is for dynamic maps that “may have any key,” and returns `nil` when it is absent.

3. How it actually works

Guards fail, they don’t raise

The docs: “In guards, when functions would normally raise exceptions, they cause the guard to fail instead.” A guard calling `map_size/1` on a list does not crash; the whole guard is simply false, and the next clause is tried. That is useful, and it is also the source of a subtle bug (section 4.2).

Truthiness

Only `false` and `nil` are falsy. Everything else is truthy, including `0`, `""`, `[]` and the string `"false"`. `&&`, `||` and `if` use truthiness; `and`, `or` and `not` require actual booleans and raise otherwise.

@enforce_keys is a construction check

`@enforce_keys` makes `%User{}` raise if a key is missing at that moment. The docs note it “is not enforced on updates and it does not provide any sort of value-validation.” Functions that build structs from data, like `struct/2`, don’t check it either; `struct!/2` does.

Big structs change representation

The runtime stores maps with up to 32 keys compactly. Above that it switches to a different internal structure, which the anti-pattern guide warns can lead to “bloating and higher memory usage” for structs; every instance of a large struct pays it.

4. Plausible but wrong

4.1 Reading required fields with []

```
def send_welcome(user) do
  Mailer.deliver(to: user[:email], name: user[:name])
end
```

If `:email` is missing, because of a typo in the key or an upstream change, this passes `nil` to the mailer, which fails somewhere far away with a confusing error. Or worse, it succeeds and sends nothing. Fix: `user.email` or a struct pattern (`%User{email: email}`), so a missing field raises `KeyError` right here. Keep `[]` for maps whose keys really are optional or dynamic.

4.2 An `or` guard that silently fails

```
def normalize(x) when map_size(x) > 0 or is_list(x), do: :ok
def normalize(_), do: :other

normalize([1])  #=> :other
```

For a list, `map_size/1` fails, which fails the *entire* guard, so the `is_list` half never counts. The code looks as if it accepts lists; it never does. Fix: separate guard clauses, each safe on its own:

```
def normalize(x) when is_map(x) and map_size(x) > 0, do: :ok
def normalize(x) when is_list(x), do: :ok
```

4.3 A variable that should have been pinned

```
expected = :ok
case do_work() do
  expected -> :success      # rebinds `expected`, matches anything
  _        -> :failure
end
```

Without `^`, `expected` in the pattern is a fresh variable that matches every value, so the function reports success for `:error` too. The only clue is a compiler warning about an unused variable. Fix: `^expected -> :success`, and treat that warning as an error.

4.4 Defensive code that hides bad data

```
def parse_pair(s) do
  case String.split(s, "=") do
    [k, v] -> {k, v}
    _      -> nil
  end
end
```

Malformed input becomes `nil`, which travels on and blows up in another module, long after the input that caused it. The anti-pattern guide calls this non-assertive pattern matching. Fix: if bad input is a bug, assert: `[k, v] = String.split(s, "=", parts: 2)`. If it is expected, return `{:error, :invalid_pair}` so the caller has to handle it (chapter 2).

4.5 Trusting `@enforce_keys`

```
defmodule User do
  @enforce_keys [:email]
  defstruct [:email, :name]
end

user = struct(User, params)  # params has no :email
```

`struct/2` does not check enforced keys, so this builds `%User{email: nil}` without complaint. The same happens with `%{user | email: nil}` or `Map.put`. Fix: build with `%User{...}` or `struct!/2`, and validate values at the boundary (a changeset or an explicit `new/1` that returns `{:ok, user} | {:error, reason}`).

4.6 Truthy strings from the environment

```
if System.get_env("FEATURE_X") do
  enable_feature_x()
end
```

With `FEATURE_X=false`, `System.get_env/1` returns the string `"false"`, which is truthy, so the feature turns on. The same happens with `"0"` or `"no"`. Fix: parse to a real boolean once, at startup, in `config/runtime.exs` (for example `System.get_env("FEATURE_X") == "true"`), and compare with `==` rather than relying on truthiness.

4.7 Defaults that override options

```
def connect(opts) do
  opts = @defaults ++ opts
  timeout = Keyword.get(opts, :timeout)
```

`Keyword` lists allow duplicate keys, and `Keyword.get/2` returns the *first*. With the defaults first, the caller's `timeout: 100` is ignored and the default of 5000 wins. Fix: `Keyword.merge(@defaults, opts)`, where the second list's values win, or `Keyword.validate!/2` (Elixir 1.13+), which also rejects unknown options.

4.8 A struct that grows past 32 fields

```
defmodule Order do
  defstruct [:id, :customer_id, :status, ...]  # 40 fields
end
```

Above 32 fields each instance uses the larger map representation, and with millions of orders in memory or in ETS the cost adds up. It usually also means the struct holds several concepts at once. Fix: group

related fields into nested structs (`%Order{shipping: %Address{}, totals: %Totals{}}`).

5. Review checklist and sources

- ❑ Required fields are read with `map.key`, pattern matching or structs, not `map[:key]`
- ❑ Guards that combine checks are safe for every input type, or split into separate clauses
- ❑ Values being compared in patterns are pinned with `^`; unused-variable warnings are errors in CI
- ❑ Unexpected data either fails fast or returns `{:error, reason}`; never a silent `nil`
- ❑ Structs from external data go through `struct!/2` or a validating constructor
- ❑ Environment strings are parsed to real types once, at startup
- ❑ Options are combined with `Keyword.merge/2` or checked with `Keyword.validate!/2`
- ❑ Structs stay under 32 fields

Sources

- Patterns and guards: <https://hexdocs.pm/elixir/patterns-and-guards.html>
- Map: <https://hexdocs.pm/elixir/Map.html>
- Keyword: <https://hexdocs.pm/elixir/Keyword.html>
- Structs: <https://hexdocs.pm/elixir/structs.html>
- Code-related anti-patterns: <https://hexdocs.pm/elixir/code-anti-patterns.html>

2. Errors and Control Flow

Chapter 2 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why Elixir has two ways to fail

Elixir separates two kinds of failure that many languages mix together:

- **Expected failures** are part of the domain: the file might not exist, the user might not be found, the input might not parse. They are *values*, usually `{:error, reason}`, and the caller decides what to do.
- **Unexpected failures** are bugs or broken assumptions: a key that must be there isn't, a function gets data it was never designed for. They *raise*, the process crashes, and a supervisor restarts it from a clean state (Erlang book, chapter 1).

Most error-handling mistakes in Elixir come from blurring the two. Rescuing an exception that signals a bug hides it. Raising for something routine makes a normal condition crash a process. Returning `nil` for either makes both invisible.

2. The building blocks

Tool	Use for
<code>{:ok, value} / {:error, reason}</code>	Expected outcomes the caller must handle
<code>foo!</code> (bang) variant	The same operation, raising instead of returning an error: for when failure means a bug
<code>case</code>	Branching on one result
<code>with</code>	A sequence of steps that each might fail
<code>raise / rescue</code>	Unexpected conditions; rescuing only at clear boundaries
<code>throw / catch</code>	Rare: escaping from code you don't control
<code>exit</code>	Process-level failure; handled by links and supervisors, not <code>try</code>

The convention the docs describe: `File.read/1` returns `{:ok, binary} | {:error, reason}`, and `File.read!/1` returns the binary or raises. Use the plain version when failure is a normal possibility, and the bang when it would mean something is broken.

with in one example

```
with {:ok, user} <- Accounts.fetch(id),
     {:ok, order} <- Orders.create(user, params),
     :ok          <- Mailer.confirm(order) do
```

```
    {:ok, order}
  end
```

Each `<-` must match or the whole `with` stops. The docs: “while `=` raises in case of not matches, `<-` will simply abort the `with` chain and return the non-matched value.”

3. How it actually works

A failed with returns whatever didn't match

Without `else`, the value that failed to match is returned as-is. If every step returns `{:error, reason}` on failure, that is exactly what you want. If one step returns something else (`false`, `nil`, `:error`), that value leaks to the caller.

else must be complete

With an `else` block, a non-matching value goes to `else`. If no `else` clause matches it, `with` raises `WithClauseError`. A partial `else` turns an unhandled error shape into a crash.

after is a soft guarantee

`try ... after` runs its cleanup when the block finishes or raises. But if the process is killed, for example by its supervisor during shutdown or by a linked crash, `after` does not run. The docs call these “soft guarantees.” Resources that must be released need an owner process whose death releases them (Erlang book, chapter 4 on ETS ownership and chapter 11 on ports).

Exceptions have a cost

Raising and rescuing captures a stacktrace and unwinds the stack. That's fine for rare events, but expensive and hard to follow as ordinary control flow. The design anti-patterns guide names this directly: *exceptions for control flow*.

4. Plausible but wrong

4.1 Parsing by catching exceptions

```
def parse_port(s) do
  try do
    {:ok, String.to_integer(s)}
  rescue
    ArgumentError -> {:error, :invalid}
  end
end
```

This is the *exceptions for control flow* anti-pattern: a routine case, bad input, goes through `raise` and `rescue`. Fix: use the non-raising API, and check the whole result, because `Integer.parse("12x")` returns

{12, "x"}}, not an error:

```
def parse_port(s) do
  case Integer.parse(s) do
    {n, ""} when n in 1..65535 -> {:ok, n}
    _ -> {:error, :invalid}
  end
end
```

4.2 A with step that doesn't return an error tuple

```
def checkout(id) do
  with {:ok, user} <- Accounts.fetch(id),
       true <- user.active do
    {:ok, place_order(user)}
  end
end
```

For an inactive user this returns `false`. The caller matches on `{:ok, _}` and `{:error, _}` and crashes with `CaseClauseError`, or worse, treats `false` as “no error”. Fix: make every step return the same shape:

```
defp ensure_active(%{active: true}), do: :ok
defp ensure_active(_), do: {:error, :inactive}
```

4.3 A partial else

```
with {:ok, user} <- fetch(id),
     true <- user.active do
  {:ok, user}
else
  {:error, reason} -> {:error, reason}
end
```

`false` goes to `else`, matches no clause, and raises `WithClauseError`. Adding `else` made the function crash instead of returning an odd value. The anti-pattern guide also warns against large `else` blocks that try to decode which step failed. Fix: normalise errors in small helpers (as in 4.2), so the `with` needs no `else` at all.

4.4 Rescuing everything

```
def handle_call({:import, rows}, _from, state) do
  result =
    try do
      {:ok, Importer.run(rows, state.schema)}
    rescue
      _ -> {:error, :import_failed}
    end
end
```

```
    {:reply, result, state}
  end
```

A `KeyError` from a bug in `Importer` now looks exactly like bad input. There is no crash report and no stacktrace, the supervisor never restarts anything, and the bug can live for months. Fix: rescue only the specific exceptions you expect and can act on, return `{:error, reason}` from `Importer` for expected failures, and let everything else crash.

4.5 A bang on user input inside a server

```
def handle_call({:load, path}, _from, state) do
  {:reply, :ok, %{state | data: File.read!(path)}}
end
```

A mistyped path from a user is an expected failure, but `File.read!/1` raises and crashes the server, losing its state and failing every other caller's request in flight. Fix: `File.read/1` and reply `{:error, reason}`. Keep bangs for things that must exist, like a bundled asset at startup.

4.6 An option that changes the return type

```
def parse(input, opts \\ []) do
  data = do_parse(input)
  if opts[:raw], do: data, else: {:ok, to_struct(data)}
end
```

Callers can't tell from the call site what shape comes back, and one missed option turns into a pattern-match error somewhere else. The design anti-patterns guide calls this *alternative return types*. Fix: two functions with fixed return types, `parse/1` and `parse_raw/1`.

4.7 Relying on after for cleanup

```
def with_lock(key, fun) do
  :ok = Locks.acquire(key)
  try do
    fun.()
  after
    Locks.release(key)
  end
end
```

If the calling process is killed while `fun` runs, for example a `Task` that times out and is shut down, `after` never runs and the lock is never released. Fix: have the lock server monitor the holder and release the lock when it gets `:DOWN`, so the lock's lifetime is tied to the process itself.

5. Review checklist and sources

- ❑ Expected failures return `{:ok, _}` / `{:error, _}`; bang functions are used only where failure means a bug
- ❑ No `try/rescue` used to parse, validate or branch on routine conditions
- ❑ Every step in a `with` returns `{:ok, _}` / `:ok` or `{:error, _}`; helpers normalise other shapes
- ❑ `with ... else` blocks are small and complete, or absent
- ❑ `rescue` names specific exceptions; no bare `rescue _`
- ❑ Options never change a function's return type
- ❑ Resources that must be released are tied to a process's lifetime, not only to `after`

Sources

- `try`, `catch`, and `rescue`: <https://hexdocs.pm/elixir/try-catch-and-rescue.html>
- `with` (Kernel.SpecialForms): <https://hexdocs.pm/elixir/Kernel.SpecialForms.html#with/1>
- Design-related anti-patterns: <https://hexdocs.pm/elixir/design-anti-patterns.html>
- Code-related anti-patterns: <https://hexdocs.pm/elixir/code-anti-patterns.html>

3. Processes, Tasks and Agents

Chapter 3 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why Elixir gives you abstractions over processes

Every concurrent thing in Elixir is a BEAM process: isolated, cheap, communicating only by messages (Erlang book, chapter 2). You can work with them directly through `spawn`, `send` and `receive`, but the Elixir guide steers you away from that: “Tasks build on top of the spawn functions to provide better error reports and introspection.”

Elixir’s standard library gives three abstractions, each shaped for one job:

- **Task:** run a computation concurrently, often to get a result back.
- **Agent:** hold a piece of state that several processes read and update.
- **GenServer:** a long-lived process with its own protocol (chapter 4).

They are thin, and that’s the point: each keeps the underlying process semantics visible. The mistakes in this chapter come from forgetting those semantics: links that propagate crashes, laziness that means nothing runs, functions that execute in a different process from the one that wrote them.

2. The building blocks

Task

Function	Linked to caller?	Result	Use for
<code>Task.async/1 + await/2</code>	Yes, and monitored	Must be awaited, default 5000 ms	Concurrent part of the caller’s own work
<code>Task.await_many/2</code>	—	Results in task order	Several <code>async</code> tasks at once
<code>Task.async_stream/3</code>	Yes	Lazy stream of <code>{:ok, v}</code> / <code>{:exit, r}</code>	Bounded concurrency over a collection
<code>Task.start_link/1</code>	Yes	None	Fire-and-forget inside a supervision tree
<code>Task.Supervisor.async_nolink/3</code>	No, monitored only	<code>{ref, result}</code> message or <code>:DOWN</code>	Work whose failure must not crash the caller
<code>Task.Supervisor.start_child/2</code>	No	None	Supervised fire-and-forget

`async_stream` defaults worth knowing: `max_concurrency` equals the number of online schedulers, results are ordered, each item has a 5000 ms `timeout`, and `on_timeout: :exit` makes a timeout exit the *caller*.

Agent

An Agent is a process holding one value. `Agent.get/2`, `update/2` and `get_and_update/2` take a function, and the docs are explicit about where it runs: “the functions passed as arguments to the calls to Agent functions are invoked inside the agent (the server).”

When to use which

Need	Use
Speed up independent work, wait for results	<code>Task.async_stream</code> or <code>async</code> + <code>await_many</code>
Work that may fail without taking you down	<code>Task.Supervisor.async_nolink</code>
Background work nobody waits for	<code>Task.Supervisor.start_child</code>
Small shared state with simple updates	Agent
State with a protocol, timers or complex rules	<code>GenServer</code> (chapter 4)
Read-mostly shared data	ETS or <code>:persistent_term</code> , not a process (Erlang book, chapter 4)

3. How it actually works

async links both ways

`Task.async` links the task to the caller. If the task crashes, the caller crashes; if the caller crashes, the task dies. Only the caller can await the result, and it must, because the reply message is always sent.

Streams are lazy

`Task.async_stream/3` returns a stream. Nothing runs until the stream is consumed by `Enum.to_list/1`, `Stream.run/1` or another `Enum` function.

Functions run where they’re sent

A function passed to an Agent runs inside the Agent’s process. A closure passed to `Task.async` runs in the task’s process, and everything it references is *copied* there (Erlang book, chapter 2). The process boundary is invisible in the syntax but decides both performance and atomicity.

Reads and writes are separate messages

`Agent.get` and `Agent.update` are two separate calls. Between them, any number of other processes can update the agent. Only a single call (`update/2` with a function, or `get_and_update/2`) is atomic.

4. Plausible but wrong

4.1 Unbounded parallelism

```
urls
|> Enum.map(fn url -> Task.async(fn -> HTTP.get(url) end) end)
|> Task.await_many(30_000)
```

With 50,000 URLs this starts 50,000 requests at once: exhausted connection pools, file descriptors and remote rate limits. It works in the demo with ten URLs. Fix: `Task.async_stream(urls, &HTTP.get/1, max_concurrency: 20, timeout: 30_000)` for concurrency with a limit.

4.2 One slow item kills the whole batch

```
results =
  items
  |> Task.async_stream(&process/1)
  |> Enum.to_list()
```

With the defaults, any item taking longer than 5 seconds makes the *caller* exit with `:timeout`, discarding every result already computed. Fix: set an explicit `timeout`, add `on_timeout: :kill_task`, and handle the resulting `{:exit, :timeout}` entries:

```
|> Task.async_stream(&process/1, timeout: 30_000, on_timeout: :kill_task)
|> Enum.flat_map(fn {:ok, r} -> [r]; {:exit, _} -> [] end)
```

4.3 A stream that never runs

```
def notify_all(users) do
  Task.async_stream(users, &Mailer.send_notice/1, max_concurrency: 10)
  :ok
end
```

This returns `:ok` and sends nothing. The stream was built and thrown away. Tests that only check the return value pass. Fix: consume it with `|> Stream.run()`, or `|> Enum.to_list()` if you need the results.

4.4 Read, then update an Agent

```
def increment(counter) do
  value = Agent.get(counter, &&1)
  Agent.update(counter, fn _ -> value + 1 end)
end
```

Other processes can update between `get` and `update`, and their changes get overwritten. In a test with 100 concurrent increments this ended with a count of `1`. Fix: do the whole change in one call: `Agent.update(counter, &(&1 + 1))`, or `Agent.get_and_update/2` when you also need the old value.

4.5 Expensive work inside the Agent

```
Agent.get(Stats, fn events -> Statistics.percentiles(events) end)
```

`percentiles/1` runs inside the Agent, so every other reader and writer waits behind it. Fix: take the data out (`Agent.get(Stats, &1)`) and compute in the caller. If most access is reads, store the data in ETS instead.

4.6 A closure that copies more than it uses

```
def handle_call({:export, id}, _from, state) do
  Task.start(fn -> Exporter.run(id, state.settings.format) end)
  {:reply, :ok, state}
end
```

The closure references `state`, so the entire server state (maybe megabytes) is copied into the new process just to read one field. The process anti-patterns guide calls this *sending unnecessary data*. Fix: extract what you need first: `format = state.settings.format`, then use `format` in the closure.

4.7 Unsupervised background work

```
def handle_cast({:reindex, id}, state) do
  Task.start(fn -> Search.reindex(id) end)
  {:noreply, state}
end
```

The task belongs to no supervision tree. At shutdown it is killed mid-write rather than stopped in order. When it crashes, nothing notices, and you can't list or limit these tasks. The process anti-patterns guide names this *unsupervised processes*. Fix: `Task.Supervisor.start_child(MyApp.TaskSup, fn -> ... end)` with the task supervisor in your application's tree.

4.8 `async_nolink` without handling its messages

```
def handle_call(:refresh, _from, state) do
  Task.Supervisor.async_nolink(MyApp.TaskSup, &fetch_rates/0)
  {:reply, :ok, state}
end
```

The task's result arrives later as a `{ref, result}` message, and a `{:DOWN, ref, ...}` message follows (or comes alone if the task crashed). Without `handle_info` clauses for them, `GenServer`'s default logs "unexpected message" warnings, and the result is lost. Fix: keep `task.ref` in the state and handle both messages:

```
def handle_info({ref, rates}, %{task_ref: ref} = s) do
  Process.demonitor(ref, [:flush])
  {:noreply, %{s | rates: rates, task_ref: nil}}
end
```

```
def handle_info({:DOWN, ref, :process, _, _reason}, %{task_ref: ref} = s),
  do: {:noreply, %{s | task_ref: nil}}
```

5. Review checklist and sources

- ☐ Concurrency over collections is bounded (`async_stream` with `max_concurrency`)
- ☐ `async_stream` has an explicit `timeout`; slow items use `on_timeout: :kill_task` where partial results are acceptable
- ☐ Every stream is consumed (`Stream.run/1` or an `Enum` function)
- ☐ Agent updates are single calls; no `get` followed by `update`
- ☐ No expensive functions run inside Agents
- ☐ Closures sent to other processes capture only the data they need
- ☐ Background work runs under a `Task.Supervisor`
- ☐ `async_nolink` results and `:DOWN` messages are handled, with `demonitor(ref, [:flush])`
- ☐ `Task.async` is used only where a task crash should crash the caller

Sources

- Processes guide: <https://hexdocs.pm/elixir/processes.html>
- Task: <https://hexdocs.pm/elixir/Task.html>
- Task.Supervisor: <https://hexdocs.pm/elixir/Task.Supervisor.html>
- Agent: <https://hexdocs.pm/elixir/Agent.html>
- Process-related anti-patterns: <https://hexdocs.pm/elixir/process-anti-patterns.html>

4. GenServer

Chapter 4 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why GenServer exists

A `GenServer` is a process that owns some state and serves requests about it, one at a time. It is Elixir’s wrapper around Erlang’s `gen_server` (Erlang book, chapter 5), and everything there still applies: calls are monitored requests with a timeout, callbacks run in sequence, and slow work in a callback blocks every caller.

The Elixir docs add one rule that matters more than any API detail: “A `GenServer`, or a process in general, must be used to model runtime characteristics of your system. A `GenServer` must never be used for code organization purposes.” Use one when you need state that outlives a call, concurrency, or a unit of failure. For anything else, write functions.

2. The building blocks

One module, two processes

By convention, a `GenServer` module contains both halves:

Part	Functions	Runs in
Client API	<code>start_link/1</code> , <code>get/1</code> , <code>put/2</code> ... wrapping <code>GenServer.call/cast</code>	The caller’s process
Server callbacks	<code>init/1</code> , <code>handle_call/3</code> , <code>handle_cast/2</code> , <code>handle_info/2</code> , <code>handle_continue/2</code> , <code>terminate/2</code>	The <code>GenServer</code> process

The module reads as one piece of code but runs in two places. Every mistake in section 4 comes from losing track of which process a line of code runs in.

Callbacks and returns

Callback	Triggered by	Common returns
<code>init/1</code>	<code>start_link</code>	<code>{:ok, state}</code> , <code>{:ok, state, :continue, term}</code>
<code>handle_call/3</code>	<code>GenServer.call/3</code> (caller waits, default 5000 ms)	<code>{:reply, reply, state}</code> , <code>{:noreply, state}</code> then <code>GenServer.reply/2</code> later
<code>handle_cast/2</code>	<code>GenServer.cast/2</code> (caller does not wait)	<code>{:noreply, state}</code>

Callback	Triggered by	Common returns
<code>handle_info/2</code>	Any other message: timers, <code>:DOWN</code> , task replies	<code>{:noreply, state}</code>
<code>handle_continue/2</code>	A <code>{:continue, term}</code> return	<code>{:noreply, state}</code>

Naming

name: value	Registered where
<code>MyServer</code> (an atom)	Locally, one per node
<code>{:global, term}</code>	Across a cluster via <code>:global</code> (Erlang book, chapter 6)
<code>{:via, Registry, {MyRegistry, key}}</code>	In a Registry: many named instances (chapter 5)

3. How it actually works

Defaults from use GenServer

`use GenServer` defines a `child_spec/1` (restart `:permanent`, shutdown 5000 ms), and default implementations of the callbacks you don't write. The default `handle_info/2` logs unexpected messages and carries on. **Once you define your own `handle_info/2`, the default is gone.** Any message your clauses don't match raises `FunctionClauseError` and crashes the server.

A call from inside the server

`GenServer.call/3` from the server to itself cannot work: the server would wait for itself. The runtime detects this and exits with `:calling_self`. The easy way to trigger it is to call the module's own client API from a callback.

Late replies

If a call times out, the caller exits (or, if it catches the exit, carries on). On OTP 24 and later the reply is sent to a process alias and discarded if it arrives late. The Elixir docs still warn that a late reply "may arrive at any time later into the caller's message queue," which is true on older OTP versions.

`handle_continue` runs before the next message

Returning `{:continue, term}` from `init/1` or any callback runs `handle_continue/2` immediately after, before the server handles anything else. That lets `init/1` return quickly, so the supervisor can move on, while expensive setup still happens before the first request.

4. Plausible but wrong

4.1 Process calls scattered through the codebase

```
# in a controller
GenServer.call(MyApp.Cart, {:add, user_id, item})
# in a background job
GenServer.cast(MyApp.Cart, {:clear, user_id})
```

Every caller knows the server’s name and its private message format. Renaming a message or moving to one server per user means editing every call site. The process anti-patterns guide calls this *scattered process interfaces*. Fix: expose client functions (`Cart.add(user_id, item)`) in the server module and keep message shapes private to it.

4.2 Expensive init

```
def init(opts) do
  rates = ExchangeAPI.fetch_all!() # 3 seconds, may fail
  {:ok, %{rates: rates, opts: opts}}
end
```

The supervisor waits for `init/1`, so application boot stalls. If the API is down, the child fails to start, and the application with it. Fix: `{:ok, %{opts: opts}, {:continue, :load}}` and fetch in `handle_continue/2`. If the data can be missing, model a “not loaded yet” state and retry with a timer.

4.3 A hard-coded name

```
def start_link(opts), do: GenServer.start_link(__MODULE__, opts, name: __MODULE__)
```

This allows exactly one instance per node. Tests that start it in parallel collide with `{:error, {:already_started, pid}}`, and a library using this pattern can’t be used twice in one application. Fix: `name: Keyword.get(opts, :name, __MODULE__)`, and have client functions take the server as an argument (defaulting to the module name).

4.4 A `handle_info` that replaces the default

```
def handle_info(:tick, n) do
  Process.send_after(self(), :tick, 1000)
  {:noreply, n + 1}
end
```

This works until any other message arrives, such as a `:DOWN` from a monitor, a stray reply, or a message from a library. Defining `handle_info/2` removed the logging default, so the server crashes with `FunctionClauseError` (verified). Fix: end with a catch-all clause that logs and keeps state:

```
def handle_info(msg, state) do
  Logger.warning("unexpected message: #{inspect(msg)}")
  {:noreply, state}
end
```

4.5 Calling your own client API from a callback

```
def handle_cast(:refresh, state) do
  current = get(:rates) # get/1 is GenServer.call(__MODULE__, ...)
  {:noreply, update(state, current)}
end
```

It reads naturally, since `get/1` is right there in the module, but the callback is already inside the server. The call exits with `:calling_self` and the server crashes (verified). Fix: inside callbacks, work on `state` directly (`Map.get(state, :rates)`); client functions are for other processes.

4.6 A periodic job that queues up

```
def handle_info(:sync, state) do
  Process.send_after(self(), :sync, 1_000)
  {:noreply, do_sync(state)} # sometimes takes 3 s
end
```

The next `:sync` is scheduled before the work runs. When the work takes longer than the interval, messages arrive faster than they are handled and the mailbox grows without bound. Fix: schedule the next run *after* the work, so the gap is measured from the end of the last run.

4.7 Returning the whole state to filter elsewhere

```
def active_sessions do
  GenServer.call(__MODULE__, :get_state)
  |> Map.values()
  |> Enum.filter(& &1.active)
end
```

Each call copies the entire state, possibly hundreds of megabytes, into the caller just to keep a few entries. The process anti-patterns guide calls this *sending unnecessary data*. Fix: filter in the server (`handle_call(:active_sessions, ...)`) and return only what was asked for. If the data is read far more than written, keep it in ETS so readers don't go through the process at all.

5. Review checklist and sources

- ☐ The GenServer models state, concurrency or failure; no GenServers for code organisation
- ☐ All interaction goes through client functions in the server's module
- ☐ `init/1` returns quickly; slow setup is in `handle_continue/2`
- ☐ Names are configurable; nothing assumes a single instance

- ❑ Every custom `handle_info/2` ends with a catch-all clause
- ❑ Callbacks never call the module's own client API
- ❑ Periodic work reschedules after it finishes
- ❑ Calls return only the data the caller needs

Sources

- GenServer: <https://hexdocs.pm/elixir/GenServer.html>
- Process-related anti-patterns: <https://hexdocs.pm/elixir/process-anti-patterns.html>
- Erlang book, chapter 5 (gen_server and gen_statem): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>

5. Supervision and Registry

Chapter 5 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why supervision is structure, not error handling

A supervisor starts processes, watches them, and restarts them from a clean state when they fail. The principles are the same as in Erlang (Erlang book, chapter 1): arrange processes so that dependencies are explicit, failures are contained, and the parts that must not fail are small.

What Elixir adds is convenience: child specs generated by `use`, the `{Module, arg}` shorthand, `DynamicSupervisor` for children started on demand, and `Registry` for finding processes by name. Convenience means defaults, and several of those defaults decide what happens in exactly the situation supervision exists for.

2. The building blocks

Child specs

`use GenServer`, `use Agent`, `use Task` and `use Supervisor` all generate `child_spec/1`, so a child list is usually just `{Module, arg}` tuples:

```
children = [
  {Registry, keys: :unique, name: MyApp.Registry},
  {DynamicSupervisor, name: MyApp.SessionSup, strategy: :one_for_one},
  {MyApp.Cache, ttl: 60}
]
Supervisor.start_link(children, strategy: :rest_for_one)
```

Key	Default	Meaning
<code>:id</code>	The module name	Must be unique within one supervisor
<code>:restart</code>	<code>:permanent</code>	<code>:permanent</code> always, <code>:transient</code> if abnormal exit, <code>:temporary</code> never
<code>:shutdown</code>	5000 for workers, <code>:infinity</code> for supervisors	How long to wait before killing the child
<code>:type</code>	<code>:worker</code>	<code>:supervisor</code> for nested supervisors

Override per module with `use GenServer, restart: :temporary`, or per use with `Supervisor.child_spec({Mod, arg}, id: :other, restart: :transient)`.

Supervisors

Kind	Children	Restart limit default	Use for
<code>Supervisor</code>	Fixed list, started in order	3 restarts in 5 s	The static structure of your application
<code>DynamicSupervisor</code>	Started on demand with <code>start_child/2</code>	3 restarts in 5 s, for <i>all</i> children together	Sessions, connections, jobs
<code>PartitionSupervisor</code> (1.14+)	N copies of a child, keyed by partition	Per partition	Spreading load or failure across several supervisors
<code>Task.Supervisor</code>	Tasks	3 restarts in 5 s	Supervised one-off work (chapter 3)

Strategies are the Erlang ones: `:one_for_one`, `:rest_for_one`, `:one_for_all`. Children start in list order and shut down in reverse.

Registry

A Registry is a local, ETS-backed key-process store. With keys: `:unique` it names processes:

```
name = {:via, Registry, {MyApp.Registry, {:session, user_id}}}
DynamicSupervisor.start_child(MyApp.SessionSup, {Session, name: name})
GenServer.call(name, :get)
```

With keys: `:duplicate` it groups processes under a key, which is the basis for local pub-sub with `Registry.dispatch/3`.

3. How it actually works

Restart limits are per supervisor

`max_restarts: 3`, `max_seconds: 5` counts restarts across *all* the supervisor's children. When the limit is exceeded the supervisor terminates every child and then itself (Erlang book, chapter 1). For a `Supervisor` with five distinct services this is sensible. For a `DynamicSupervisor` with thousands of independent children, it means a handful of unrelated crashes takes down every one of them.

Registry cleanup is eventually consistent

When a registered process dies, the Registry removes its entries, but not atomically with the death. The docs warn that a pid can die “at any time, including between getting the value from the registry and sending it a message.” A lookup can return a dead pid, and a new process registering the same key immediately may briefly see the old entry.

Order is dependency

Children start strictly in order, and each child's `init/1` finishes before the next starts. A child that uses the Registry must come after the Registry. With `:rest_for_one`, a restart of the Registry also restarts everything after it, which is usually exactly what dependents need.

4. Plausible but wrong

4.1 A few crashes take down every session

```
children = [{DynamicSupervisor, name: SessionSup, strategy: :one_for_one}]
# ... thousands of sessions via DynamicSupervisor.start_child(SessionSup, {Session, id})
```

Sessions are `:permanent` by default, and the supervisor allows 3 restarts in 5 seconds for all of them. In a test with 1,000 children, 4 crashes from bad input killed the `DynamicSupervisor` and **all 1,000** children (verified). Fix: make per-session processes `restart: :temporary` (clients reconnect), and if some must restart, raise `max_restarts` deliberately or spread them with a `PartitionSupervisor` so failures stay contained.

4.2 Two children with the same id

```
children = [{Worker, :orders}, {Worker, :payments}]
Supervisor.start_link(children, strategy: :one_for_one)
```

Both children get `id: Worker`, and the supervisor refuses to start (verified: `{:error, ...}`). Under an application this fails the whole boot with an error about duplicate child specifications. Fix: `Supervisor.child_spec({Worker, :orders}, id: :orders_worker)` for each.

4.3 Using the Registry before it exists

```
children = [
  {DynamicSupervisor, name: SessionSup},
  {SessionPreloader, []}, # starts sessions named via Registry
  {Registry, keys: :unique, name: MyApp.Registry}
]
```

`SessionPreloader` starts before the Registry, so its via-tuple names fail and boot crashes. It works in a test that starts the Registry separately. Fix: put the Registry first and use `:rest_for_one`, so a Registry restart also restarts everything that depends on it.

4.4 Assuming a lookup finds a live process

```
def notify(user_id, msg) do
  [{pid, _}] = Registry.lookup(MyApp.Registry, {:session, user_id})
```

```
    send(pid, msg)
  end
```

If the user has no session, the match raises. If the session just died, the lookup may still return its pid, and `send/2` silently goes nowhere. Fix: handle `[]` explicitly. For requests, call through the via-tuple (`GenServer.call({:via, Registry, ...}, msg)`), which exits with `:noproc` when nobody is there, instead of a silent send.

4.5 Restarting into an old registration

```
def restart_session(user_id) do
  [{pid, _}] = Registry.lookup(MyApp.Registry, {:session, user_id})
  :ok = DynamicSupervisor.terminate_child(SessionSup, pid)
  DynamicSupervisor.start_child(SessionSup, {Session, name: via(user_id)})
end
```

Because Registry cleanup is eventually consistent, the new process can find the old registration still present and fail with `{:error, {:already_registered, old_pid}}`. It depends on timing, so it passes locally and fails under load. Fix: treat `already_registered` as a normal outcome (retry briefly, or reuse the existing process), or design so that the same process is kept rather than replaced.

4.6 Supervising per-request processes as permanent

```
defmodule Upload do
  use GenServer      # :permanent by default
  def init(%{socket: s} = st), do: {:ok, st}
end
```

When an upload crashes because the client vanished, the supervisor restarts it with the same arguments, including a socket that no longer exists. It crashes again straight away, consuming the restart budget (4.1). Fix: use `GenServer`, `restart: :temporary` for anything tied to an external session; use `:transient` for jobs that should retry after failure but not after finishing normally.

4.7 Cleanup that never runs at shutdown

```
def terminate(_reason, state) do
  flush_buffer(state)
end
```

`terminate/2` runs on supervisor shutdown only if the process traps exits (Erlang book, chapter 1, section 4.1). Without that, the buffer is lost on every deploy. Fix: `Process.flag(:trap_exit, true)` in `init/1`, a `:shutdown` value large enough for the flush, and remember that `terminate/2` is still not guaranteed if the process is killed.

5. Review checklist and sources

- ❑ Children are ordered by dependency; Registries come before their users; `:rest_for_one` where later children depend on earlier ones
- ❑ Every child in a supervisor has a unique `:id`
- ❑ Per-connection and per-request processes are `restart: :temporary`
- ❑ `DynamicSupervisor` restart limits are set deliberately for the number of children, or children are spread with `PartitionSupervisor`
- ❑ Registry lookups handle `[]` and dead pids; requests go through via-tuples
- ❑ Re-registering a key handles `{:error, {:already_registered, _}}`
- ❑ Processes with cleanup trap exits and have a suitable `:shutdown`

Sources

- Supervisor: <https://hexdocs.pm/elixir/Supervisor.html>
- `DynamicSupervisor`: <https://hexdocs.pm/elixir/DynamicSupervisor.html>
- `PartitionSupervisor`: <https://hexdocs.pm/elixir/PartitionSupervisor.html>
- Registry: <https://hexdocs.pm/elixir/Registry.html>
- Erlang book, chapter 1 (Supervision Trees): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>

6. Enum, Stream and Collections

Chapter 6 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why collections are where performance hides

Most Elixir code is pipelines over collections: `Enum.map`, `Enum.filter`, `Enum.reduce`. They read the same whether the data has ten elements or ten million, which is exactly the problem. A line that is instant in a test can be quadratic in production, and nothing in the syntax tells you.

Two facts explain nearly every collection bug in Elixir:

1. **Lists are singly linked.** Reaching the front is instant; reaching the n th element, the end, or the length means walking the whole list.
2. **Enum is eager and Stream is lazy.** Every `Enum` step builds a full intermediate result. A `Stream` builds a recipe that runs only when something consumes it, and runs again every time something consumes it.

2. The building blocks

Cost of common operations on a list

Operation	Cost	Note
Prepend, <code>hd/1</code> , head-tail pattern match	Constant	Always prefer working at the front
<code>length(list)</code> , <code>Enum.count(list)</code>	Linear	Don't use to test for emptiness
<code>Enum.at(list, i)</code>	Linear in <code>i</code>	Lists are not arrays
<code>list ++ other</code>	Linear in <code>list</code>	Copies the left side
<code>List.last(list)</code>	Linear	
<code>Enum.reverse(list)</code>	Linear, one pass	Cheap way to finish a prepend-based build

For indexed access use a tuple (`elem/2`) or a map keyed by index; for sets use `MapSet`; for queues use `:queue`.

Enum or Stream?

Use Enum when	Use Stream when
The data fits comfortably in memory	The data is large or unbounded
You need the whole result	You need only part of it (<code>Enum.take/2</code> , <code>Enum.find/2</code>)

Use Enum when	Use Stream when
The default	Reading from slow sources: files (<code>File.stream!/1</code>), sockets, paginated APIs

The docs recommend starting with `Enum` and moving to `Stream` only when “laziness is required.”

Comprehensions and early exit

- `for x <- xs, filter, into: %{} build`s any collectable; `reduce`: turns it into a fold.
- `Enum.reduce_while/3` stops as soon as the function returns `{:halt, acc}`.
- `Enum.find/2`, `Enum.any?/2` and `Enum.take/2` stop early too.

3. How it actually works

Streams are recipes, not data

A `Stream` value holds the source and the chain of functions. Each `Enum` call on it runs the whole chain from the start. Side effects in the chain (logging, HTTP calls, counters, reading a file) happen again every time the stream is consumed.

Maps are not ordered

Maps with up to 32 keys happen to iterate in sorted key order; larger maps use a hash structure with no useful order. Neither follows insertion order. Code that relies on map order works by accident, and breaks when the map grows past 32 keys or the keys change.

Ranges pick their own direction

`first..last` without an explicit step counts *down* when `first > last`. The docs call this behaviour deprecated and recommend explicit steps: `1..n//1` is empty when `n` is 0, and `3..1// -1` for real decreasing ranges.

4. Plausible but wrong

4.1 Indexing a list in a loop

```
for i <- 0..(length(items) - 1) do
  process(Enum.at(items, i), i)
end
```

Each `Enum.at/2` walks from the start, so the loop is quadratic. With 20,000 items, compiled code took **356 ms**, against **1.2 ms** for `Enum.map/2` over the same list. Fix: `Enum.with_index(items)` and iterate once:

```
for {item, i} <- Enum.with_index(items), do: process(item, i)
```

4.2 Appending in a reduce

```
Enum.reduce(rows, [], fn row, acc -> acc ++ [transform(row)] end)
```

`++` copies the accumulator on every step. For 20,000 rows this took **842 ms**, against **0.2 ms** for prepending and reversing once (measured with compiled code). Fix: `Enum.map(rows, &transform/1)`, or prepend with `[x | acc]` and call `Enum.reverse/1` at the end.

4.3 length for emptiness

```
if length(queue) > 0, do: dispatch(queue)
```

This counts every element to answer a yes-or-no question. On a long queue checked often it becomes a hot spot. Fix: `queue != []`, or match in a function head: `def dispatch([_ | _] = queue).`

4.4 A range that runs backwards

```
def retry(n) do
  for attempt <- 1..n, do: try_once(attempt)
end
retry(0) # runs attempts 1 and 0
```

With `n = 0` the range is `1..0`, which counts down, so the loop runs twice instead of not at all (verified: `[1, 0]`). Off-by-one bugs like this are easy to miss because they only show at the boundary. Elixir 1.18 prints a deprecation warning when such a range is built at run time, but the loop still runs (verified). Fix: `1..n//1`, which is empty when `n < 1`.

4.5 A stream consumed twice

```
events = Stream.map(ids, &Api.fetch_event!/1)
total = Enum.count(events)
first_ten = Enum.take(events, 10)
```

The stream runs once per consumer: every event is fetched for `count`, then the first ten are fetched again (verified: side effects ran twice). With a file stream, the file is re-read; with a paginated API, you pay twice and may get different data. Fix: materialise once with `Enum.to_list/1` when the data fits, or restructure into a single pass.

4.6 Relying on map order

```
headers = Map.keys(row) |> Enum.join(",")
values = Map.values(row) |> Enum.join(",")
```

The column order is sorted, not the order you wrote the keys (verified: `%{"zeta" => 1, "alpha" => 2}` gives alpha first). Once a row has more than 32 fields, the order is effectively arbitrary. Fix: keep an

explicit list of columns and read values with `Map.fetch!/2` in that order, or use a keyword list when order is part of the data.

4.7 Loading everything to use a little

```
File.read!("access.log")
|> String.split("\n")
|> Enum.filter(&String.contains?(&1, "500"))
|> Enum.take(5)
```

For a multi-gigabyte log this reads and splits the whole file into memory to return five lines. Fix:

`File.stream!("access.log") |> Stream.filter(...) |> Enum.take(5)` reads line by line and stops after the fifth match.

5. Review checklist and sources

- ☐ No `Enum.at/2` inside loops over lists; `Enum.with_index/1` or a different structure instead
- ☐ No `acc ++ [x]` in reductions; `prepend` and `reverse`, or use `Enum.map/2`
- ☐ Emptiness is tested with `== []` or pattern matching, not `length/1`
- ☐ Ranges that can be empty use an explicit step (`1..n//1`)
- ☐ Streams with side effects are consumed exactly once
- ☐ No code depends on map iteration order
- ☐ Large or slow sources are processed with `Stream` and stop early where possible
- ☐ `Enum` is the default; `Stream` is used for a reason

Sources

- Enumerables and streams: <https://hexdocs.pm/elixir/enumerable-and-streams.html>
- Enum: <https://hexdocs.pm/elixir/Enum.html>
- Stream: <https://hexdocs.pm/elixir/Stream.html>
- Range: <https://hexdocs.pm/elixir/Range.html>
- List: <https://hexdocs.pm/elixir/List.html>

7. Strings and Unicode

Chapter 7 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why “string” means three different things

An Elixir string is a UTF-8 encoded binary. That one sentence hides three separate ways of counting:

- **Bytes:** what the binary actually holds, what a database column limit or a network protocol counts.
- **Code points:** Unicode characters: `é` can be one code point, or two (`e` plus a combining accent).
- **Graphemes:** what a person sees as one character: `👍` is one grapheme made of two code points and eight bytes.

`String` functions work in graphemes; `byte_size/1`, `binary_part/3` and pattern matching with `<<...>>` work in bytes. Most string bugs come from counting in one unit and limiting in another. English test data hides them, because for ASCII all three counts are the same.

Elixir also has *charlists*, lists of code points (`~c"abc"`, or `'abc'` in older code), used mainly to talk to Erlang libraries.

2. The building blocks

Function	Counts or works in	Cost
<code>byte_size/1</code>	Bytes	Constant
<code>String.length/1</code>	Graphemes	Linear: walks the whole string
<code>String.slice/3</code> , <code>String.at/2</code>	Graphemes	Linear
<code>binary_part/3</code> , <code><<head::binary-size(n), _::binary>></code>	Bytes	Constant; can cut a character in half
<code>String.graphemes/1</code> / <code>String.codepoints/1</code>	Graphemes / code points	Linear
<code>String.valid?/1</code>	Checks the bytes are valid UTF-8	Linear
<code>String.normalize/2</code> , <code>String.equivalent?/2</code>	Unicode normal forms (NFC, NFD...)	Linear
<code>String.upcase/2</code> , <code>downcase/2</code>	Graphemes, with <code>:default</code> , <code>:ascii</code> , <code>:turkic</code> , <code>:greek</code> modes	Linear

Strings versus charlists

	String	Charlist
Literal	<code>"abc"</code>	<code>~c"abc"</code> (older code: <code>'abc'</code>)

	String	Charlist
Representation	UTF-8 binary	List of integer code points
Memory	About 1 byte per ASCII character	2 words (16 bytes) per character (Erlang book, chapter 3)
Where you meet it	Elixir code	Erlang APIs (<code>:inet</code> , <code>:io_lib</code> , <code>:os.cmd...</code>)
Convert	<code>List.to_string/1</code>	<code>String.to_charlist/1</code>

3. How it actually works

Equal-looking is not equal

Unicode allows the same visible text to be encoded in different ways. "café" typed on one keyboard can be `c a f é` (one code point for `é`), while text from another system can be `c a f e ΅` (e plus a combining accent). They look identical, but `==` compares bytes and returns `false`. `String.equivalent?/2` compares normalised forms; `String.normalize(s, :nfc)` at the boundary makes all later comparisons reliable.

Regular expressions default to bytes

Elixir regexes are PCRE. Without the `u` modifier, `\w`, `.` and character classes work on bytes and ASCII rules, so `~r/^\w+$/` does not match "café". With `~r/^\w+$/u` it does.

An integer list may print as text

`I0.inspect/1` prints a list of integers as a charlist when all of them are printable. `[7, 8, 9]` prints as `~c"\a\b\t"` (`'\a\b\t'` on older versions). Nothing is wrong with the data; only the display changed. Pass `charlists: :as_lists` to `inspect` to see numbers.

4. Plausible but wrong

4.1 Limiting bytes with String.length

```
def fits_column?(name), do: String.length(name) <= 255
def truncate(name), do: String.slice(name, 0, 255)
```

The database limit is 255 *bytes*; this checks graphemes. A name of 150 `é` characters passes the check at 150 graphemes but is 300 bytes (verified), and the insert fails, or the column silently truncates it. Fix: check `byte_size/1`. To truncate safely, cut on a grapheme boundary: take graphemes while the running byte total stays under the limit.

4.2 Cutting bytes in the middle of a character

```
preview = binary_part(body, 0, min(byte_size(body), 151))
```

Taking a fixed number of bytes can split a multi-byte character, leaving invalid UTF-8 (verified: `String.valid?/1` returns `false`). JSON encoders then raise, and some clients display garbage. Fix: truncate by graphemes (`String.slice/3`) when the limit is visual, or back off to a valid boundary when the limit is in bytes.

4.3 Comparing text from different sources

```
def same_city?(a, b), do: String.downcase(a) == String.downcase(b)
```

"café" from a web form and "café" from an import can differ in encoding, so this returns `false` for identical-looking values (verified: `== false`, `String.equivalent?/2 true`). Duplicates slip past uniqueness checks. Fix: normalise all incoming text once, with `String.normalize(s, :nfc)`, before storing or comparing.

4.4 ASCII-only regex on Unicode input

```
def valid_username?(u), do: Regex.match?(~r/^\w{3,20}$/u, u)
```

"josé" and every non-Latin name are rejected, because without `u` the regex works on bytes with ASCII rules (verified). Fix: `~r/^\w{3,20}$/u`. Decide deliberately which characters you allow: `\w` with `u` accepts letters from every script.

4.5 Joining an Erlang charlist with <>

```
{:ok, {a, b, c, d}} = get_peer_ip(socket)
"client " <> :inet.ntoa({a, b, c, d})
```

`:inet.ntoa/1` returns a charlist, `~c"10.0.0.1"`, and `<>` requires binaries, so this raises `ArgumentError` at runtime (verified). String interpolation hides the problem because it converts charlists automatically, so the same value works in one place and fails in another. Fix: convert at the boundary: `:inet.ntoa(ip) |> List.to_string()`.

4.6 String.length as an emptiness test

```
def blank?(s), do: String.length(String.trim(s)) == 0
```

`String.length/1` walks the entire string to count graphemes. On large inputs checked often, that is wasted work. Fix: `String.trim(s) == ""`, or `byte_size(s) == 0` for an exact empty check.

4.7 Upper-casing for keys and comparisons

```
key = String.upcase(code)    # "istanbul" -> "ISTANBUL"
```

Default Unicode rules are right for most languages but not all: Turkish distinguishes dotted and dotless i, and `String.upcase/2` has a `:turkic` mode for that. Case-folding user text to build lookup keys can merge or split words unexpectedly. Fix: for identifiers you control, restrict input to ASCII and use `String.upcase(code, :ascii)`. For human text, compare normalised strings and choose the mode deliberately.

5. Review checklist and sources

- ☐ Byte limits (database columns, protocols) are checked with `byte_size/1`, not `String.length/1`
- ☐ Truncation never splits a character; results are valid UTF-8
- ☐ Incoming text is normalised (NFC) before storage or comparison
- ☐ Regexes that see user text use the `u` modifier
- ☐ Charlists from Erlang APIs are converted with `List.to_string/1` at the boundary
- ☐ Emptiness is tested with `== ""`, not `String.length/1`
- ☐ Case conversion for keys uses an explicit mode (`:ascii` for identifiers)

Sources

- String: <https://hexdocs.pm/elixir/String.html>
- Binaries, strings, and charlists: <https://hexdocs.pm/elixir/binaries-strings-and-charlists.html>
- Regex: <https://hexdocs.pm/elixir/Regex.html>
- Erlang book, chapter 3 (Binaries and Memory): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>

8. Protocols and Behaviours

Chapter 8 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why Elixir has two kinds of polymorphism

Elixir has no classes, but it has two ways to write code that works across different things:

- A **protocol** dispatches on the *data type* of the first argument. `Enumerable`, `String.Chars` (`to_string/1`) and `Inspect` are protocols: `Enum.map/2` works on lists, maps, ranges and your own structs because each has an implementation. Anyone can add an implementation for a new type, without touching the protocol.
- A **behaviour** is a contract for a *module*: a list of callbacks the module promises to define. `GenServer`, `Supervisor` and `Application` are behaviours; you pick the module explicitly (`GenServer.start_link(MyServer, ...)`), and the behaviour's generic code calls your callbacks.

The short rule: vary by **data**, use a protocol; vary by **implementation you choose**, use a behaviour (plus configuration or dependency injection).

2. The building blocks

Protocols

```
defprotocol Shape do
  @spec area(t) :: number
  def area(shape)
end

defimpl Shape, for: Circle do
  def area(%Circle{r: r}), do: :math.pi() * r * r
end
```

Feature	Meaning
<code>defimpl P, for: Type</code>	Implementation for a built-in type or a struct
<code>@derive [P] / @derive {P, opts}</code>	Generate an implementation for a struct from the protocol's <code>Any</code> implementation
<code>@fallback_to_any true</code>	Use the <code>Any</code> implementation for every type without its own (the docs prefer explicit <code>@derive</code>)
<code>Protocol.UndefinedError</code>	Raised when a type has no implementation
Consolidation	Mix compiles all implementations into fast dispatch at build time

Structs do not inherit map implementations: a struct is not `Enumerable` and does not support `Access` unless you implement them.

Behaviours

```
defmodule Notifier do
  @callback deliver(user :: map, message :: String.t()) :: :ok | {:error, term}
end

defmodule EmailNotifier do
  @behaviour Notifier
  @impl true
  def deliver(user, msg), do: ...
end
```

Feature	Meaning
@callback	Declares a required function and its typespec
@behaviour Mod	Declares that this module implements it; missing callbacks produce warnings
@impl true	Marks a function as a callback; the compiler checks it matches one
@optional_callbacks	Callbacks a module may skip
use Mod	Often sets @behaviour and injects default implementations

3. How it actually works

use injects defaults, so a typo is never missing

`use GenServer` defines default versions of callbacks you don't write. If you misspell one (`handle_cal/3`), your function is just an ordinary extra function, and the default handles the real callback: `GenServer.call/2` fails at runtime with "no `handle_call/3` clause was provided" (verified). Nothing warns at compile time, unless you mark callbacks with `@impl true`. Then the typo produces a warning: "got `@impl true` for function `handle_cal/3` but no behaviour specifies such callback" (verified).

Inspect shows everything by default

The default `Inspect` implementation for a struct prints every field. Logs, crash reports, `IO.inspect` and error trackers all use it. Structs holding secrets need an explicit `@derive {Inspect, only: ...}` or `except: ...`.

Dispatch is on the first argument only

A protocol picks its implementation from the type of the first argument, nothing else. For behaviour that depends on two types, or on configuration, a protocol is the wrong tool.

4. Plausible but wrong

4.1 Secrets in logs

```
defmodule Account do
  defstruct [:email, :password_hash, :api_token]
end

Logger.error("sync failed for #{inspect(account)}")
```

Default `inspect` prints `password_hash` and `api_token` into the log, and from there into log storage, error trackers and support tickets (verified). Fix: `@derive {Inspect, except: [:password_hash, :api_token]}`, which prints `#Account<email: "a@b.c", ...>` (verified). Better still, use `only:` to list the fields that are safe.

4.2 A misspelled callback

```
defmodule Pinger do
  use GenServer
  def init(s), do: {:ok, s}
  def handle_cal(:ping, _from, s), do: {:reply, :pong, s}
end
```

It compiles without a word. The first `GenServer.call(pid, :ping)` crashes the server with “no `handle_call/3` clause was provided”. Fix: put `@impl true` on every callback, so a misspelled name produces a compile-time warning. Once one callback in a module has `@impl`, the compiler also warns about callbacks that lack it, which keeps the habit consistent.

4.3 Enum over a struct

```
def total(%Order{} = order) do
  Enum.reduce(order, 0, fn {_k, v}, acc -> acc + v end)
end
```

Structs are not `Enumerable`, so this raises `Protocol.UndefinedError` (verified). A test with a plain map passes. Fix: iterate over the field you mean (`order.items`). If you really need the fields, use `Map.from_struct/1`, and question whether you do.

4.4 A fallback that hides missing implementations

```
defprotocol Billing.Price do
  @fallback_to_any true
  def amount(item)
end

defimpl Billing.Price, for: Any do
  def amount(_), do: 0
end
```

Every new product type without an implementation is now free, silently. The `Protocol.UndefinedError` you would have got in the first test is gone. Fix: no fallback for anything with business meaning. If a default is really correct, opt in per struct with `@derive Billing.Price`.

4.5 Unrelated behaviour in one multi-clause function

```
def update(%Product{} = p, attrs), do: Catalog.save(p, attrs)
def update(%Invoice{} = i, attrs), do: Accounting.amend!(i, attrs)
def update(%User{} = u, attrs), do: Accounts.change_email(u, attrs.email)
```

Three different operations, with different rules and failure modes, share a name. The design anti-patterns guide calls this *unrelated multi-clause function*: documentation, specs and callers can't describe it honestly. Fix: separate, named functions. If the operation really is the same idea across types, a protocol makes it explicit and extensible.

4.6 Choosing an implementation by building a module name

```
def notifier(kind) do
  Module.concat([MyApp.Notifiers, Macro.camelize(kind)])
end
notifier(params["channel"]).deliver(user, msg)
```

User input decides which module is called, atoms are created from input (chapter 1), and the compiler can't see the dependency. The macro anti-patterns guide calls this *untracked compile-time dependencies*. Fix: an explicit map from allowed values to modules:

```
@notifiers %{"email" => EmailNotifier, "sms" => SmsNotifier}
def notifier(kind), do: Map.fetch(@notifiers, kind)
```

4.7 Defining an implementation at runtime

```
# in a test helper or a script loaded after compilation
defimpl String.Chars, for: Money do
  def to_string(m), do: "#{m.amount} #{m.currency}"
end
```

Protocols are consolidated at build time. An implementation defined later is not part of the consolidated dispatch table, so it behaves differently in tests, `ix` and releases. Fix: keep every `defimpl` in `lib/`, so it is compiled with the project. For test-only structs, put them in `test/support` compiled via `elixirc_paths` in `mix.exs`.

5. Review checklist and sources

- ☐ Protocols are used for data-driven polymorphism; behaviours for chosen implementations
- ☐ Every callback is marked `@impl true`
- ☐ Structs with secrets derive `Inspect` with `only:` or `except:`
- ☐ No `Enum` functions on structs; iterate the intended field
- ☐ No `@fallback_to_any` for protocols with business meaning
- ☐ Multi-clause functions group one operation, not several unrelated ones
- ☐ Implementations are chosen from explicit maps, not modules built from strings
- ☐ All `defimpls` live in compiled project code

Sources

- Protocols: <https://hexdocs.pm/elixir/protocols.html>
- Typespecs and behaviours: <https://hexdocs.pm/elixir/typespecs.html>
- Inspect: <https://hexdocs.pm/elixir/Inspect.html>
- Design-related anti-patterns: <https://hexdocs.pm/elixir/design-anti-patterns.html>
- Meta-programming anti-patterns: <https://hexdocs.pm/elixir/macro-anti-patterns.html>

9. Macros

Chapter 9 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why macros exist, and why to avoid them

A macro is a function that runs at compile time, receives code as data (an AST), and returns new code to put in its place. Much of Elixir itself is macros: `if`, `def`, `|>`, `use`, and the test macros in ExUnit. They let libraries like Ecto and Phoenix offer syntax that reads like a language of its own.

The docs are blunt about using them yourself: “Macros should only be used as a last resort,” because “explicit is better than implicit” and “clear code is better than concise code.” A macro makes code harder to read, to debug and to compile quickly, and its mistakes don’t look like mistakes at the call site. Official anti-pattern pages cover five macro-specific problems. Assistants reach for macros readily, because they are impressive, so this chapter is about recognising when one isn’t needed and getting it right when it is.

2. The building blocks

Tool	Meaning
<code>quote do ... end</code>	Turn code into its AST without running it
<code>unquote(x)</code>	Insert a value or AST into a quote
<code>quote bind_quoted: [v: expr]</code>	Evaluate <code>expr</code> once, bind it to <code>v</code> inside the quote
<code>defmacro</code>	Define a macro; arguments arrive as AST
<code>require Mod</code>	Needed before calling <code>Mod</code> ’s macros
<code>use Mod, opts</code>	Calls <code>Mod.__using__(opts)</code> and inserts whatever code it returns
<code>var!(x)</code>	Deliberately break hygiene to reach a caller’s variable
<code>Macro.to_string/1</code> , <code>Macro.expand/2</code>	Inspect what a macro generates

Macro or function?

Use a function when	A macro may be justified when
The inputs are values	You need the <i>code</i> of an argument (to show it in an error, like <code>assert</code>)
Behaviour happens at run time	Work must happen at compile time (generating functions from a schema)
You want to call it dynamically or pass it around	You are building a DSL whose syntax is the point

3. How it actually works

Arguments are code, not values

A macro receives the AST of its arguments. Every `unquote(arg)` in the result places that *code* in the output, so if the macro uses an argument twice, the expression runs twice. `bind_quoted` avoids this by evaluating each argument once, at the start of the generated code.

Hygiene

Variables created inside a `quote` belong to the macro's context, not the caller's. A macro that writes `x = 1` does not define `x` for the code around it. That is usually what you want; `var!/1` overrides it, and makes the macro's effect harder to see.

Compile-time dependencies

When a module uses a macro, the modules referenced in the macro's arguments can become *compile-time* dependencies: changing one forces recompiling the other. In a large project a few such macros can make every small edit recompile hundreds of files. `mix xref graph --label compile-connected` shows these chains.

use can inject anything

`use Mod` runs `Mod.__using__/1`, which can import functions, alias modules, define functions, set attributes and register callbacks, none of it visible at the call site. The anti-pattern guide recommends `import` and `alias` instead, where they are enough.

4. Plausible but wrong

4.1 A macro that should be a function

```
defmacro cents(amount) do
  quote do: round(unquote(amount) * 100)
end
```

There is nothing here a function can't do: no code inspection, no compile-time work. It can't be passed as `¢s/1`, needs `require`, and makes stack traces point at generated code. The anti-pattern guide calls this *unnecessary macros*. Fix: `def cents(amount), do: round(amount * 100)`.

4.2 Evaluating an argument twice

```
defmacro positive!(expr) do
  quote do
    if unquote(expr) > 0, do: unquote(expr), else: raise("not positive")
  end
end
```

```
positive!(next_id())
```

`next_id()` is called twice: once for the check, once for the result. With a counter it returned 2 and was called twice (verified). With a database insert or an HTTP call, the side effect happens twice. Fix: `quote bind_quoted: [value: expr]` and use `value`. That call returned 1, called once (verified).

4.3 Expecting a macro to set a caller's variable

```
defmacro load_config, do: quote(do: config = Application.get_all_env(:my_app))

def start do
  load_config()
  connect(config)
end
```

Hygiene keeps `config` inside the macro. The caller's `config` doesn't exist, and compilation fails with an undefined `config` (verified with a similar macro). The tempting fix, `var!(config)`, compiles but hides a variable binding inside a macro call. Fix: return a value (`config = load_config()`), and at that point a function will do.

4.4 Runtime values read at compile time

```
defmacro api_url do
  url = System.get_env("API_URL")
  quote do: unquote(url)
end
```

Code outside `quote` runs when the *caller* is compiled, so the URL from the build machine's environment is baked into the `.beam`, and production silently uses it. Fix: keep runtime lookups inside the generated code (or better, in a plain function), and read environment variables in `config/runtime.exs` (chapter 10).

4.5 use as a grab bag

```
defmodule MyApp.Web do
  defmacro __using__(_) do
    quote do
      import MyApp.Helpers
      import MyApp.Formatting
      alias MyApp.{Repo, Accounts, Billing}
      require Logger
    end
  end
end
```

Every module that says `use MyApp.Web` silently gets dozens of imports and aliases. Name clashes appear far from their cause, and readers can't tell where a function comes from. The anti-pattern guide calls this

“use” instead of “import”. Fix: explicit `import / alias` in each module; keep `use` for cases where code must genuinely be injected, and document what it injects.

4.6 Large generated code

```
defmacro defroute(path, handler) do
  quote do
    def handle(unquote(path), conn) do
      # 60 lines of validation, parsing and error handling
    end
  end
end
```

Every call copies all 60 lines into the caller. A few hundred routes means tens of thousands of generated lines, slow compiles and large beams. The anti-pattern guide calls this *large code generation*. Fix: generate only the dispatch clause, and call an ordinary function for the work: `def handle(unquote(path), conn), do: Router.run(unquote(handler), conn)`.

4.7 Module names built at compile time from strings

```
for name <- ~w(user order invoice) do
  mod = Module.concat([MyApp.Schemas, Macro.camelize(name)])
  def schema(unquote(name)), do: unquote(mod)
end
```

The compiler can't see that this module depends on `MyApp.Schemas.User` and the others, so changes don't trigger the right recompilation, and missing modules surface only at runtime. The anti-pattern guide calls this *untracked compile-time dependencies*. Fix: write the module names out (`def schema("user"), do: MyApp.Schemas.User`) or keep an explicit map.

5. Review checklist and sources

- ☐ Every macro needs compile-time behaviour or the code of its arguments; otherwise it is a function
- ☐ Macros bind their arguments once (`bind_quoted`, or `unquote` each argument exactly once)
- ☐ No macro relies on setting the caller's variables; `var!` is justified where used
- ☐ No environment or configuration is read outside `quote` in a macro
- ☐ `use` is reserved for real code injection and documents what it injects; otherwise `import/alias`
- ☐ Generated code is small and delegates to functions
- ☐ Module names are written explicitly; compile-time dependency chains are checked with `mix xref`

Sources

- Macros: <https://hexdocs.pm/elixir/macros.html>
- Quote and unquote: <https://hexdocs.pm/elixir/quote-and-unquote.html>
- Meta-programming anti-patterns: <https://hexdocs.pm/elixir/macro-anti-patterns.html>
- Kernel.SpecialForms (quote): <https://hexdocs.pm/elixir/Kernel.SpecialForms.html#quote/2>

10. Mix, Configuration and Releases

Chapter 10 · checked against Elixir 1.20 · Sep 26, 2026

1. Why build time and run time must stay apart

An Elixir project goes through two separate lives. At **build time**, Mix compiles your code on a build machine, with that machine’s environment, and assembles a release. At **run time**, the release boots on a server, with a different environment, and no Mix at all.

Configuration bugs almost always come from code that runs in one life and assumes the other. A value read during compilation is frozen into the `.beam` file. A call to Mix works on your laptop and doesn’t exist in production. A dependency marked for development compiles fine in `mix test` and is missing from the release. The Erlang book covers applications and releases from the OTP side (chapter 7); this chapter covers the Elixir tooling on top of it.

2. The building blocks

Configuration files

File	Evaluated	Can read	Use for
<code>config/config.exs</code> (+ <code>dev.exs</code> , <code>test.exs</code> , <code>prod.exs</code>)	At build time, before compiling	Build machine’s environment	Settings that shape compilation; per- environment defaults
<code>config/runtime.exs</code>	Each time the system or release starts	The target machine’s environment	Secrets, URLs, ports, anything deployment- specific

The docs: `config.exs` “is read at build time, before we compile our application,” while `runtime.exs` “is read after our application and dependencies are compiled.”

Reading configuration

Function	When it reads	Notes
<code>Application.get_env/3</code> , <code>fetch_env!/2</code>	When called	Use inside functions for runtime values
<code>Application.compile_env/3</code> , <code>compile_env!/2</code>	At compile time	Mix records the value and raises at boot if runtime config differs
<code>System.get_env/1</code>	When called	Returns <code>nil</code> if unset
<code>System.fetch_env!/1</code>	When called	Raises if unset: use in <code>runtime.exs</code> for required values

Dependencies

Option	Effect
<code>only: [:dev, :test]</code>	Not fetched or compiled for other environments; absent from prod releases
<code>runtime: false</code>	Compiled, but the application is not started at runtime (for build tools)
<code>mix.lock</code>	Records exact versions; commit it so every build uses the same code

Releases

`mix release` builds a self-contained directory with your code, dependencies and the Erlang runtime. The docs state the rule plainly: release code “MUST NOT access Mix in any way, as Mix is a build tool and it is not available inside releases.”

3. How it actually works

Module bodies run at compile time

Everything at the top level of a module, including module attributes, runs when the module is compiled. `@timeout Application.get_env(:app, :timeout)` stores whatever the config said on the build machine. The docs warn that in module bodies “the application environment is not yet available,” and point to `compile_env/3` when a compile-time value really is intended.

compile_env checks itself

`Application.compile_env/3` records the value used during compilation, and at boot “Mix will... compare the compilation values with the runtime values whenever your system starts, raising an error in case they differ.” A mismatch fails loudly at startup instead of silently in production.

The application environment is global

Every application’s configuration lives in one global store per node. The docs recommend that libraries avoid it: “the application environment is effectively a global storage.” Two dependents of a library can’t configure it differently.

4. Plausible but wrong

4.1 Secrets in build-time config

```
# config/prod.exs
config :my_app, MyApp.Repo, url: System.get_env("DATABASE_URL")
```

This runs on the build machine. There, `DATABASE_URL` is usually unset, so the release ships with `nil`, or with the build machine's database URL baked in. Fix: move it to `config/runtime.exs` and use `System.fetch_env!("DATABASE_URL")` so a missing value stops the boot with a clear error.

4.2 Configuration frozen in a module attribute

```
defmodule MyApp.Client do
  @base_url Application.get_env(:my_app, :base_url)
  def get(path), do: HTTP.get(@base_url <> path)
end
```

`@base_url` is evaluated at compile time, so setting `base_url` in `runtime.exs` changes nothing. Fix: read it at runtime (`Application.fetch_env!(:my_app, :base_url)` inside the function). If it truly must be compile-time, use `Application.compile_env!/2`, which at least raises at boot when runtime config disagrees.

4.3 Calling Mix at runtime

```
def debug?, do: Mix.env() == :dev
```

It works in `iex -S mix` and in tests. In a release the `Mix` module doesn't exist, and the first call raises `UndefinedFunctionError`. Fix: decide at build time with configuration: `config :my_app, debug: true` in `dev.exs`, read with `Application.get_env/3`.

4.4 A dev-only dependency used in production code

```
{:jason, "~> 1.4", only: [:dev, :test]}

def to_json(data), do: Jason.encode!(data)
```

Everything passes in development and CI. The prod release doesn't include `jason`, and the first call fails with `UndefinedFunctionError`. Fix: include every dependency that `lib/` code calls in all environments, and build and smoke-test the actual release in CI.

4.5 A library configured through the application environment

```
defmodule MyLib.Client do
  def request(path) do
    key = Application.fetch_env!(:my_lib, :api_key)
    ...
  end
end
```

Every application that depends on `my_lib` must share one global setting, so two services in one release can't use different keys, and tests can't run in parallel with different values. The design anti-patterns guide calls this *using application configuration for libraries*. Fix: accept options in function calls or `start_link/1`, and let the host application pass them from its own config.

4.6 Required values that default to nil

```
# config/runtime.exs
config :my_app, :payment_key, System.get_env("PAYMENT_KEY")
```

A typo in the variable name or a missing deployment secret gives `nil`. The system boots, reports healthy, and fails at the first payment. Fix: `System.fetch_env!/1` for required values, and parse types here too (`String.to_integer/1` for ports, explicit `== "true"` for flags, chapter 1).

4.7 An uncommitted lock file

```
.gitignore:
mix.lock
```

Without the lock file, each build resolves dependency versions afresh. The laptop, CI and production can all run different versions of the same library, and a patch release upstream changes your system without a commit. Fix: commit `mix.lock`; update dependencies deliberately with `mix deps.update` and review the diff.

5. Review checklist and sources

- ☐ Everything environment-specific is read in `config/runtime.exs`
- ☐ Required values use `System.fetch_env!/1`; types are parsed at startup
- ☐ No `Application.get_env` in module attributes; `compile_env` is used where compile-time values are intended
- ☐ No `Mix` calls in `lib/` code
- ☐ Dependencies used by `lib/` code are available in all environments
- ☐ Libraries take configuration as options, not from the application environment
- ☐ `mix.lock` is committed
- ☐ CI builds and boots the release, not only `mix test`

Sources

- Configuration and releases: <https://hexdocs.pm/elixir/config-and-releases.html>
- Application: <https://hexdocs.pm/elixir/Application.html>
- mix release: <https://hexdocs.pm/mix/Mix.Tasks.Release.html>
- Design-related anti-patterns: <https://hexdocs.pm/elixir/design-anti-patterns.html>
- Erlang book, chapter 7 (Applications and Releases): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>

11. Testing with ExUnit

Chapter 11 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why testing concurrent code needs its own habits

ExUnit makes tests easy to write and fast to run: `async: true` runs test modules in parallel, `assert_receive` checks messages, `start_supervised!` manages processes, and doctests turn documentation into tests. The general principles (test the unhappy paths, use properties, don't restate the implementation) are in the Erlang book, chapter 8.

Elixir-specific trouble comes from concurrency inside the test suite itself. Tests that pass one at a time fail when run together, or pass today and fail on a loaded CI machine. The culprits are almost always shared state, fixed timeouts that are too short or too long, and processes that outlive or escape the test that started them.

2. The building blocks

Tool	Meaning
<code>use ExUnit.Case, async: true</code>	This module's tests run concurrently with other async modules; tests in one module still run one at a time
<code>async: true, group: :name (1.18+)</code>	Modules in the same group never run concurrently
<code>setup / setup_all</code>	Per test (in the test process) / once per module (in a separate process)
<code>start_supervised!/1</code>	Start a process under the test's supervisor; stopped when the test ends
<code>assert_receive pattern, timeout</code>	Wait for a message; default timeout 100 ms
<code>refute_receive pattern, timeout</code>	Check no message arrives within the timeout; also 100 ms by default
<code>capture_log/1, @tag capture_log: true</code>	Capture log output
<code>doctest Mod</code>	Run the <code>iex></code> examples in <code>Mod</code> 's documentation
Test timeout	60 seconds per test by default

For mocking, the standard library is Mox. Its rules: mocks are defined from behaviours (chapter 8), expectations are private to the test process by default, and child processes need `allow/3`. Processes started with `Task` are allowed automatically through `$callers`. Global mode is "incompatible with `async: true`."

3. How it actually works

async means other modules run at the same time

With `async: true`, your module runs at the same time as other async modules. Anything global is shared across all of them: registered names, the application environment, ETS tables with fixed names, Logger configuration, the file system, a real database without a sandbox. The docs: async “should be enabled only if tests do not change any global state.”

Short default timeouts cut both ways

`assert_receive` waits only 100 ms by default, which is too short for anything that crosses a slow CI machine. `refute_receive` also waits only 100 ms, so it can pass for a message that arrives 200 ms later. Both kinds of error are silent in fast local runs.

The test process is special

Processes started with `start_link` inside a test are linked to the test process and die with it, while `start_supervised!/1` stops them in order and waits. Code running in *other* processes (a GenServer under test) doesn't carry the test's Mox expectations unless allowed.

4. Plausible but wrong

4.1 Async tests that share a named process

```
defmodule CacheTest do
  use ExUnit.Case, async: true
  test "caches values" do
    {:ok, _} = MyApp.Cache.start_link([]) # registers name: MyApp.Cache
    ...
  end
end
```

Run alongside another async module that starts the same named server, and one of them fails with `{:error, {:already_started, pid}}`, depending on scheduling. In a test with three such modules, two failed (verified). Fix: accept a `:name` option (chapter 4, section 4.3) and start a uniquely named instance per test, with `start_supervised!({MyApp.Cache, name: unique_name})`, or make the module `async: false`.

4.2 Async tests that change the application environment

```
test "strict mode rejects bad input" do
  Application.put_env(:my_app, :mode, :strict)
  assert {:error, _} = MyApp.validate(bad_input)
end
```

The application environment is global. Another async test setting `:mode` to `:lenient` in the meantime makes this fail with the other test's value (verified: left: `:lenient`, right: `:strict`). Fix: pass options explicitly to the code under test (chapter 10, section 4.5). Where that's impossible, run the module with `async: false` and restore the value in `on_exit/1`.

4.3 refute_receive that proves nothing

```
test "does not email unconfirmed users" do
  Signup.register(unconfirmed_user)
  refute_receive {:email_sent, _}
end
```

`refute_receive` waits 100 ms. If the email is sent 300 ms later by a background job, the test still passes, and the bug ships (verified: the message arrived after `refute_receive` had already passed). Fix: synchronise on a point after which the email would already have been sent, such as a `call` to the process that sends it or an explicit completion message, then `refute_received` (no wait) or `refute_receive` with a justified timeout.

4.4 assert_receive that fails under load

```
Worker.process_async(job)
assert_receive {:done, ^job}
```

The work takes 300 ms on a busy CI runner and the default 100 ms timeout fails the test (verified: “no matching message after 100ms”). It passes locally, so it looks flaky rather than wrong. Fix: give `assert_receive` a timeout that reflects the operation (`assert_receive {:done, ^job}, 2_000`); a long timeout on `assert_receive` costs nothing when the message arrives quickly.

4.5 Sleeping instead of synchronising

```
Counter.increment_async(pid)
Process.sleep(50)
assert Counter.get(pid) == 1
```

Too short on a slow machine, wasted time on a fast one. Fix: the call is the synchronisation. Because messages from one process arrive in order, `Counter.get/1` (a call) is handled after the cast (Erlang book, chapter 8, section 3). Drop the sleep.

4.6 A mock called from another process

```
test "fetches rates" do
  expect(RatesMock, :fetch, fn -> {:ok, %{usd: 1.0}} end)
  {:ok, pid} = start_supervised(RateServer)
  assert RateServer.current(pid) == %{usd: 1.0}
end
```

`RateServer` calls `RateMock.fetch/0` from its own process, which is not the test process and not a `Task`, so `Mox` raises an unexpected call error there. The server crashes, and the failure appears as a confusing exit. Fix: `Mox.allow(RateMock, self(), pid)` after starting the server, or inject the implementation into the server and test its logic as plain functions.

4.7 Doctests with non-deterministic output

```
@doc """
    iex> MyApp.Session.new("ada")
    %MyApp.Session{user: "ada", id: "4f9c...", started_at: ~U[2026-09-26 10:00:00Z]}
    """
```

The id and timestamp differ on every run, and so do pids, references and the order of large maps. The docs also warn that doctests don't capture side effects and run without a sandbox. Fix: show deterministic parts only (`iex> MyApp.Session.new("ada").user / "ada"`), and keep effectful behaviour in ordinary tests.

5. Review checklist and sources

- ☐ `async: true` modules touch no global state: no fixed names, application env, named ETS tables or shared files
- ☐ Processes in tests start with `start_supervised!/1` and unique names
- ☐ `assert_receive` timeouts reflect the real operation; `refute_receive` follows an explicit synchronisation point
- ☐ No `Process.sleep/1` used to wait for work
- ☐ Mocks are behaviour-based; processes that use them are allowed (`Mox.allow/3`) or receive injected implementations
- ☐ Doctests are deterministic and free of side effects
- ☐ Expected values come from requirements, not from running the implementation (Erlang book, chapter 8)

Sources

- ExUnit.Case: https://hexdocs.pm/ex_unit/ExUnit.Case.html
- ExUnit.Assertions: https://hexdocs.pm/ex_unit/ExUnit.Assertions.html
- ExUnit.DocTest: https://hexdocs.pm/ex_unit/ExUnit.DocTest.html
- Mox: <https://hexdocs.pm/mox/Mox.html>
- Erlang book, chapter 8 (Testing): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>

12. Types

Chapter 12 · checked against Elixir 1.20 · Sep 26, 2026

1. Why types in Elixir are in transition

Elixir is dynamically typed: values carry their types at run time, and pattern matching and guards check them where it matters. For years, the only static checking came from *typespecs* plus Dialyzer, an Erlang tool that reports provable contradictions and nothing else (Erlang book, chapter 8).

That is changing. Elixir is gaining a *gradual set-theoretic type system* built into the compiler. The work began landing in the 1.17 era and has grown with each release; version 1.20 infers types from guards, from how function bodies use their arguments, and across `case`, `cond` and `with` clauses. The docs describe it as sound, gradual and still a milestone in progress: today it only *infers*; user-written type signatures are planned but “absent.”

So a codebase today has two separate type notations with different jobs, and it is easy to believe one of them is doing something it isn’t.

2. The building blocks

What checks what

Mechanism	Written by you?	Checked by	Catches	Misses
Pattern matching and guards	Yes	The runtime, every call	Wrong shapes, at the point of entry	Nothing that passes the pattern
Typespecs (<code>@spec</code> , <code>@type</code>)	Yes	Nothing in the compiler; Dialyzer if you run it	Contradictions Dialyzer can prove	Anything merely <i>possible</i> ; bugs hidden by specs that are too narrow
Set-theoretic inference (1.17+, much expanded in 1.20)	No: inferred	The compiler, on every build	Operations that can never succeed on the inferred types	Anything typed <code>dynamic()</code> ; it is best-effort
<code>@enforce_keys</code> , <code>struct!/2</code>	Yes	Runtime, at construction	Missing keys	Wrong value types (chapter 1)

Typespecs in brief

```
@type status :: :active | :suspended
@type t :: %__MODULE__ {id: pos_integer(), status: status()}

@spec suspend(t()) :: {:ok, t()} | {:error, :already_suspended}
```

The docs are clear about their role: typespecs are for documentation and for tools like Dialyzer, and “may be phased out as the set-theoretic type effort moves forward.” The compiler does not use them to check or optimise code.

3. How it actually works

Specs are claims, not checks

A typespec is never enforced at run time. `@type t :: %User{age: integer()}` does not stop `%User{age: "42"}` from existing. Dialyzer only reports cases where your code *cannot* satisfy the spec, and it trusts specs that overlap with what it infers, so a spec that is too narrow can silence real problems (Erlang book, chapter 8, section 4.5).

Inference is gradual

The new type system gives values it can't pin down the type `dynamic()`, and doesn't complain about them. Within a project, calls between your own modules are treated as `dynamic()` for now (to avoid cascading recompilation), and the docs note it “doesn't guarantee it will find all possible type incompatibilities” and can produce false positives in some constructs. A clean compile is evidence, not proof.

Runtime checks are still the contract

Because neither system is complete, the guarantee at a module's boundary is still what its patterns and guards enforce. Data from outside (JSON, forms, databases) must be validated where it enters.

4. Plausible but wrong

4.1 Treating a spec as validation

```
@spec create(%{name: String.t(), age: non_neg_integer()}) :: {:ok, User.t()}
def create(params), do: {:ok, struct!(User, params)}
```

The spec says `age` is a non-negative integer, but nothing checks it. `%{"age" => "-5"}` from a form flows straight into the struct. Fix: validate at the boundary (a changeset, or explicit guards and conversions) and return `{:error, reason}` for bad input. Keep the spec as documentation of the validated result.

4.2 A spec narrower than the code

```
@spec fetch(id()) :: User.t()
def fetch(id) do
  case Repo.get(User, id) do
    nil -> {:error, :not_found}
    user -> user
  end
end
```

```
end
end
```

The spec omits `{:error, :not_found}`. Dialyzer accepts it because the types overlap, then analyses callers as if errors can't happen, so a caller that correctly handles the error can get a “pattern can never match” warning, inviting someone to delete it. Fix: write the real return type, `User.t() | {:error, :not_found}`, and treat narrowing specs to silence warnings as a bug.

4.3 Specs made of `term()`

```
@spec process(term(), keyword()) :: term()
```

This passes every check and documents nothing. Generated code often looks like this because it is always “correct.” Fix: specify what the function actually takes and returns, even approximately (`map()`, `{:ok, Order.t()} | {:error, atom()}`), or leave the spec out rather than pretend.

4.4 Trusting a clean compile

```
def label(%{status: status}), do: "Status: " <> status # status is an atom
```

Whether the compiler warns here depends on what it can infer about the map at the call sites. If the value comes from another module in your project, or from decoded JSON, it is `dynamic()` and there is no warning. The bug appears at run time as `ArgumentError`. Fix: tests for the unhappy paths (chapter 11) and explicit conversions (`Atom.to_string/1` or interpolation), not reliance on the checker.

4.5 A growing Dialyzer ignore file

```
.dialyzer_ignore.exs: 140 entries
```

Every entry was added to make CI green “for now.” Dialyzer warnings are almost never false alarms, because it only reports what it can prove. The ignore file becomes a list of unexamined bugs. Fix: fix or justify each entry with a comment, and fail CI on new warnings.

4.6 Opaque types used as open maps

```
@opaque t :: %{items: list(), total: integer()}
...
# in another module
cart.total
```

An `@opaque` type promises that other modules won't look inside. Reaching in works at run time, and Dialyzer reports it. It also ties the other module to the internal shape, which `@opaque` exists to prevent. Fix: expose functions (`Cart.total(cart)`), or make the type public with `@type` if the structure really is the API.

5. Review checklist and sources

- ❑ External data is validated at the boundary; specs are not relied on as checks
- ❑ Specs describe the real inputs and outputs, including every error shape
- ❑ No blanket `term()` specs on public functions
- ❑ Compiler type warnings are treated as errors in CI; a clean compile is not treated as proof
- ❑ Dialyzer runs in CI; ignore entries are few and justified
- ❑ `@opaque` types are only accessed through their module's functions

Sources

- Gradual set-theoretic types: <https://hexdocs.pm/elixir/gradual-set-theoretic-types.html>
- Typespecs reference: <https://hexdocs.pm/elixir/typespecs.html>
- Elixir changelog: <https://hexdocs.pm/elixir/changelog.html>
- Dialyzer User's Guide: https://www.erlang.org/doc/apps/dialyzer/dialyzer_chapter.html
- Erlang book, chapter 8 (Testing, on Dialyzer): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>

13. Observability

Chapter 13 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why observability in Elixir is mostly the BEAM's

Elixir runs on a runtime you can inspect from the inside: list processes, read their state, trace calls, measure schedulers, all on a live node (Erlang book, chapter 9). Elixir adds two things on top that most applications use every day:

- **Logger**, Elixir's interface to Erlang's `:logger`, with metadata and macros that skip work when a level is disabled.
- **:telemetry**, the ecosystem-wide library through which Phoenix, Ecto, Oban, Broadway and others publish events: request durations, query times, job results. Metrics, dashboards (Phoenix LiveDashboard) and tracing integrations all subscribe to these events.

Both are easy to use in ways that cost performance or lose exactly the information you need during an incident.

2. The building blocks

Logger

Feature	Meaning
<code>Logger.debug/info/warning/error(...)</code>	Macros; the message is evaluated only if the level is enabled
<code>Logger.debug(fn -> ... end)</code>	Explicitly lazy message
<code>Logger.metadata(request_id: id)</code>	Attach key-value context to every later log line in this process
Structured reports	Log a map or keyword list; formatters and handlers keep the fields
<code>compile_time_purge_matching</code>	Remove chosen log calls at compile time
Erlang <code>:logger</code> underneath	Handlers, filters and overload protection (Erlang book, chapter 9)

Telemetry

Function	Meaning
<code>:telemetry.execute(event, measurements, metadata)</code>	Emit an event, e.g. <code>[:my_app, :checkout, :stop]</code>

Function	Meaning
<code>:telemetry.span(event, meta, fun)</code>	Emit <code>:start</code> and <code>:stop</code> (or <code>:exception</code>) events with durations around <code>fun</code>
<code>:telemetry.attach(id, event, handler, config)</code>	Subscribe a handler function to an event

The telemetry docs stress one point: “The `handle_event` callback of each handler is invoked synchronously on each `telemetry:execute` call.” Handlers run inside the process that emits the event, in the middle of its work.

3. How it actually works

Logger macros skip work, but only for their own arguments

Because `Logger.debug/1` is a macro, `Logger.debug("state: #{inspect(state)}")` checks the level before building the string. With debug disabled, the interpolation never runs (verified). But if you build the message in a variable first, it is built regardless of the level (verified).

Metadata lives in the process

`Logger.metadata/1` stores context in the current process. New processes start with none: inside a `Task.async` spawned from a request, `Logger.metadata()` is empty (verified), so the task’s log lines have no `request_id`.

Handlers can detach themselves

If a telemetry handler raises, telemetry detaches it (so the failure can’t repeat on every event) and logs a warning. The metric it was feeding silently stops updating until the application restarts, while the rest of the system works normally.

Handlers cost the emitter

Everything a handler does, like formatting, writing to a socket, or calling a `GenServer`, adds latency to the request, query or job that emitted the event. A slow metrics backend becomes slow requests.

4. Plausible but wrong

4.1 Building the log message before logging

```
msg = "processing order #{inspect(order, limit: :infinity)}"  
Logger.debug(msg)
```

The message is built before `Logger.debug/1` can check the level, so with debug disabled in production the full `inspect` still runs on every order (verified). Fix: put the expression inside the call (`Logger.debug("processing order #{order.id}")`) or pass a function. Log identifiers, not whole structures.

4.2 Losing request context in spawned work

```
def create(conn, params) do
  Logger.metadata(request_id: conn.assigns.request_id)
  Task.async(fn -> Billing.charge(params) end) |> Task.await()
end
```

Log lines from `Billing.charge/1` run in the task, which has no metadata (verified), so they can't be tied to the request. That is exactly what you need when a charge fails. Fix: capture and set it in the new process:

```
md = Logger.metadata()
Task.async(fn -> Logger.metadata(md); Billing.charge(params) end)
```

4.3 A telemetry handler that does I/O

```
:telemetry.attach("audit", [:my_app, :order, :stop], fn _e, m, meta, _ ->
  HTTPClient.post!(@audit_url, %{order: meta.order_id, ms: m.duration})
end, nil)
```

The HTTP call runs inside every order request, synchronously. When the audit service slows down, checkout slows down, and when it fails, the handler raises and is detached (4.4). Fix: keep handlers cheap. Update a counter or ETS table, or send a message to a process that batches and ships the data.

4.4 A handler that raises once and disappears

```
def handle_event([:repo, :query], %{total_time: t}, %{source: source}, _) do
  Metrics.observe("db.query", t, table: String.to_existing_atom(source))
end
```

The first query without a `:source` (a raw SQL query, a migration) crashes the handler. Telemetry detaches it and logs one warning, and database metrics stop for the rest of the node's life while dashboards show a quiet, healthy flat line. Fix: match defensively on metadata in handlers (a final clause that ignores unexpected shapes), alert on missing metrics, and check logs for detach warnings.

4.5 Anonymous functions as handlers

```
:telemetry.attach("req", [:phoenix, :endpoint, :stop], fn _, m, _, _ ->
  Metrics.observe("http", m.duration)
end, nil)
```

It works, but the telemetry docs recommend function captures (`&MyApp.Telemetry.handle_event/4`) for performance: they are cheaper to call on every event, and appear by name in traces and warnings. An anonymous function defined in `lex` or a script also doesn't survive code reloading the way a module function does. Fix: define handlers as public functions in a module and attach with `&Module.fun/4`.

4.6 Logging instead of measuring

```
Logger.info("checkout took #{duration} ms")
```

Latency hidden in log lines can't be graphed, aggregated or alerted on without parsing text, and under load logger overload protection may drop some lines (Erlang book, chapter 9, section 4.6). Fix: emit a telemetry event (`:telemetry.execute([:my_app, :checkout, :stop], %{duration: d}, meta)`), feed it to metrics, and keep logs for events you need to read.

4.7 Debug tooling left on in production

```
:sys.trace(MyApp.OrderServer, true)    # "just for a minute"
```

`:sys.trace/2` prints every event of that process, and `dbg`-style tracing on busy functions can flood a node (Erlang book, chapter 9, section 4.3). Left on, it fills logs and slows the process. Fix: use time-boxed or rate-limited tools (`recon_trace`), and turn tracing off explicitly (`:sys.trace(pid, false)`, `:sys.no_debug/1`), as the GenServer docs also advise.

5. Review checklist and sources

- ☐ Log messages are built inside the Logger call (or a function), never beforehand
- ☐ Logs carry identifiers, not whole structs or payloads
- ☐ Request metadata is copied into tasks and other spawned processes
- ☐ Telemetry handlers are cheap, never do I/O, and hand work off to other processes
- ☐ Handlers tolerate unexpected metadata; detach warnings are monitored
- ☐ Handlers are attached as `&Module.fun/4` captures
- ☐ Durations and counts are telemetry metrics, not log lines
- ☐ Tracing in production is time-boxed and switched off explicitly

Sources

- Logger: <https://hexdocs.pm/logger/Logger.html>
- telemetry: <https://hexdocs.pm/telemetry/readme.html>
- GenServer (debugging with `:sys`): <https://hexdocs.pm/elixir/GenServer.html>
- Erlang book, chapter 9 (Observability): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>

14. Review Cheat Sheet

Chapter 14 · all checklists in one place · Sep 26, 2026

How to use this sheet

This is every chapter's checklist, regrouped by the question a reviewer asks. The number after each item is the chapter that explains it. Start with the red flags to find suspicious code quickly, then use the sections to review it properly. For runtime topics (memory, ETS, distribution, schedulers), use the cheat sheet in the companion *Learning Erlang from Claude* as well.

1. Red flags you can search for

These patterns are not always wrong, but each one deserves a second look.

Search for	Why it's suspicious	Ch.
<code>map[:field]</code> on data with required fields	Missing keys become <code>nil</code> far from the cause	1
<code>or</code> combining type checks in a guard	One failing check fails the whole guard	1
<code>if System.get_env(...)</code>	<code>"false"</code> is truthy	1
<code>defaults ++ opts</code>	The first duplicate key wins; caller options ignored	1
<code>struct(Mod, params)</code>	Ignores <code>@enforce_keys</code>	1
<code>rescue _ -></code> or <code>rescue</code> around parsing	Hides bugs; exceptions as control flow	2
<code>true <-</code> or <code>false <-</code> in <code>with</code>	Leaks a boolean instead of an error tuple	2
<code>Task.async</code> inside <code>handle_*</code> callbacks	Linked crash plus blocked server	3
<code>Task.async_stream(...)</code> without <code>Stream.run/Enum</code>	Never runs	3
<code>Agent.get</code> followed by <code>Agent.update</code>	Lost updates	3
<code>Task.start(</code> outside a supervisor	Unsupervised process	3
<code>name: __MODULE__</code> hard-coded	Single instance; test collisions	4, 11
<code>GenServer.call(__MODULE__</code> called from a callback	Exits with <code>:calling_self</code>	4

Search for	Why it's suspicious	Ch.
<code>def handle_info</code> without a catch-all clause	Crashes on stray messages	4
<code>DynamicSupervisor</code> with <code>:permanent</code> children	A few crashes kill every child	5
<code>[{pid, _}] = Registry.lookup(</code>	Crashes on no match; pid may be dead	5
<code>Enum.at(</code> inside a loop	Quadratic	6
<code>acc ++ [</code> in a reduce	Quadratic	6
<code>length(list) > 0</code>	Walks the whole list	6
<code>for i <- 1..n</code> where n can be 0	Counts down: runs twice	6
<code>String.length(</code> for byte limits	Graphemes are not bytes	7
<code>~r/.../</code> without <code>u</code> on user text	ASCII-only rules	7
Structs with password, token or key fields and no <code>@derive Inspect</code>	Secrets in logs	8
<code>use GenServer</code> without <code>@impl true</code>	Misspelled callbacks compile silently	8
<code>@fallback_to_any true</code>	Missing implementations become silent defaults	8
<code>Module.concat / String.to_atom</code> to pick modules	Untracked dependencies; atoms from input	8, 9
<code>defmacro</code> whose body a function could do	Unnecessary macro	9
<code>unquote(expr)</code> twice in one quote	Evaluates the argument twice	9
<code>System.get_env</code> in <code>config/*.exs</code> other than <code>runtime.exs</code>	Read on the build machine	10
<code>@attr Application.get_env(</code>	Frozen at compile time	10
<code>Mix.env()</code> in <code>lib/</code>	<code>Mix</code> doesn't exist in releases	10
<code>only: [:dev, :test]</code> on a dependency used in <code>lib/</code>	Missing from the release	10
<code>Application.put_env</code> in <code>async: true</code> tests	Shared global state	11
<code>refute_receive</code> right after an <code>async</code> action	Passes for late messages	11
<code>Process.sleep</code> in tests	Synchronising by waiting	11
<code>@spec ... :: term()</code>	Documents nothing	12

Search for	Why it's suspicious	Ch.
Log message built into a variable, then <code>Logger.debug(msg)</code>	Built even when debug is off	13
HTTP or <code>GenServer.call</code> in a telemetry handler	Runs synchronously in the emitter	13

2. Is the data what we think it is?

- ❑ Required fields are read with `map.key`, pattern matching or structs, not `map[:key]` (1)
- ❑ Guards that combine checks are safe for every input type, or split into separate clauses (1)
- ❑ Values compared in patterns are pinned with `^`; unused-variable warnings are errors in CI (1)
- ❑ Structs from external data go through `struct!/2` or a validating constructor (1)
- ❑ External data is validated at the boundary; specs are not relied on as checks (12)
- ❑ Environment strings are parsed to real types once, at startup (1, 10)
- ❑ Options are combined with `Keyword.merge/2` or checked with `Keyword.validate!/2` (1)
- ❑ Structs stay under 32 fields (1)

3. How are failures handled?

- ❑ Expected failures return `{:ok, _}` / `{:error, _}`; bang functions only where failure means a bug (2)
- ❑ Unexpected data fails fast or returns `{:error, reason}`, never a silent `nil` (1)
- ❑ No `try/rescue` for parsing, validation or routine branching (2)
- ❑ Every `with` step returns `:ok`, `{:ok, _}` or `{:error, _}`; `else` blocks are small and complete, or absent (2)
- ❑ `rescue` names specific exceptions (2)
- ❑ Options never change a function's return type (2)
- ❑ Resources that must be released are tied to a process's lifetime, not only to `after` (2)

4. Are processes used for the right reasons?

- ❑ Processes model state, concurrency or failure; no GenServers for code organisation (4)
- ❑ All interaction goes through client functions in the server's module (4)
- ❑ Callbacks never call the module's own client API (4)
- ❑ Calls return only the data the caller needs; closures capture only what they use (3, 4)
- ❑ No expensive functions run inside Agents; Agent updates are single calls (3)
- ❑ `Task.async` only where a task crash should crash the caller; otherwise `Task.Supervisor.async_nolink` (3)
- ❑ `async_nolink` results and `:DOWN` messages are handled, with `demonitor(ref, [:flush])` (3)

5. What happens under load?

- ❑ Concurrency over collections is bounded (`async_stream` with `max_concurrency`) (3)

- ❑ `async_stream` has an explicit timeout; `on_timeout: :kill_task` where partial results are fine (3)
- ❑ `init/1` returns quickly; slow setup is in `handle_continue/2` (4)
- ❑ Periodic work reschedules after it finishes (4)
- ❑ No `Enum.at/2` in loops, no `acc ++ [x]`, no `length/1` for emptiness (6)
- ❑ Large or slow sources use `Stream` and stop early; `Enum` is the default otherwise (6)
- ❑ Streams with side effects are consumed exactly once; every stream is consumed (3, 6)

6. What happens when something crashes?

- ❑ Children are ordered by dependency; Registries come before their users; `:rest_for_one` where needed (5)
- ❑ Every child in a supervisor has a unique `:id` (5)
- ❑ Per-connection and per-request processes are `restart: :temporary` (5)
- ❑ `DynamicSupervisor` restart limits suit the number of children, or `PartitionSupervisor` spreads them (5)
- ❑ Background work runs under a `Task.Supervisor` (3)
- ❑ Every custom `handle_info/2` ends with a catch-all clause (4)
- ❑ Processes with cleanup trap exits and have a suitable `:shutdown` (5)

7. Can processes be found reliably?

- ❑ Names are configurable; nothing assumes a single instance (4)
- ❑ Registry lookups handle `[]` and dead pids; requests go through via-tuples (5)
- ❑ Re-registering a key handles `{:error, {:already_registered, _}}` (5)

8. Is text handled correctly?

- ❑ Byte limits use `byte_size/1`; truncation never splits a character (7)
- ❑ Incoming text is normalised (NFC) before storage or comparison (7)
- ❑ Regexes on user text use the `u` modifier (7)
- ❑ Erlang charlists are converted with `List.to_string/1` at the boundary (7)
- ❑ Emptiness is tested with `== ""`; case conversion for keys uses an explicit mode (7)
- ❑ No code depends on map iteration order (6)
- ❑ Ranges that can be empty use an explicit step (`1..n//1`) (6)

9. Is polymorphism and metaprogramming justified?

- ❑ Protocols for data-driven polymorphism, behaviours for chosen implementations (8)
- ❑ Every callback is marked `@impl true` (8)
- ❑ Structs with secrets derive `Inspect` with `only:` or `except:` (8)
- ❑ No `Enum` on structs; no `@fallback_to_any` with business meaning (8)
- ❑ Multi-clause functions group one operation; implementations come from explicit maps (8)
- ❑ All `defimpls` live in compiled project code (8)
- ❑ Every macro needs compile-time behaviour or its arguments' code; otherwise it is a function (9)

- ❑ Macros bind arguments once, don't set caller variables, and don't read config outside `quote` (9)
- ❑ `use` injects only what must be injected and documents it; generated code is small (9)
- ❑ Module names are explicit; compile-time dependencies are checked with `mix xref` (9)

10. Will it work as a release?

- ❑ Everything environment-specific is read in `config/runtime.exs`; required values use `System.fetch_env!/1` (10)
- ❑ No `Application.get_env` in module attributes; `compile_env` where compile-time values are intended (10)
- ❑ No `Mix` calls in `lib/` (10)
- ❑ Dependencies used by `lib/` are available in all environments; `mix.lock` is committed (10)
- ❑ Libraries take configuration as options, not from the application environment (10)
- ❑ CI builds and boots the release (10)

11. Do the tests prove anything?

- ❑ `async: true` modules touch no global state (11)
- ❑ Test processes use `start_supervised!/1` and unique names (11)
- ❑ `assert_receive` timeouts fit the operation; `refute_receive` follows a synchronisation point (11)
- ❑ No `Process.sleep/1` to wait for work (11)
- ❑ Mocks are behaviour-based and allowed in the processes that use them (11)
- ❑ Doctests are deterministic and side-effect free (11)
- ❑ Expected values come from requirements, not the implementation (11)

12. Do the types tell the truth?

- ❑ Specs describe real inputs and outputs, including every error shape; no blanket `term()` (12)
- ❑ Compiler type warnings fail CI; a clean compile is not treated as proof (12)
- ❑ Dialyzer runs in CI with few, justified ignores; `@opaque` types are respected (12)

13. Can you see what it's doing?

- ❑ Log messages are built inside the `Logger` call; logs carry identifiers, not payloads (13)
- ❑ Request metadata is copied into tasks and spawned processes (13)
- ❑ Telemetry handlers are cheap, tolerant of unexpected metadata, and attached as `&Module.fun/4` (13)
- ❑ Durations and counts are metrics, not log lines; detach warnings are monitored (13)
- ❑ Production tracing is time-boxed and switched off explicitly (13)